

n8n Automation Mastery

من الصفر إلى بناء أنظمة أتمتة احترافية

الإصدار المرجعي: n8n 2.37.10

تاريخ البناء: 05-09-2026

المستوى: مبتدئ ← Automation Architect

الوحدات: 15 وحدة 50 مستويات 60 مشاريع

n8n Automation Mastery

من الصفر إلى بناء أنظمة أتمتة احترافية

الإصدار المرجعي: n8n 2.37.10 · تاريخ التحقق: 5 سبتمبر 2026 المستوى: من الصفر إلى Automation Architect الأسلوب: Learn →

Build → Test → Debug → Improve

#	القسم	المستوى
0	كيف تستخدم هذا المنهج	—
1	Module 1 — مقدمة في الأتمتة	Foundations
2	Module 2 — واجهة n8n ونموذج التنفيذ	Foundations
3	Module 3 — Triggers	Foundations
4	Module 4 — البيانات و JSON	Data Fluency
5	Module 5 — Expressions	Data Fluency
6	Module 6 — Core Nodes	Data Fluency
7	Module 7 — APIs	Integration
8	Module 8 — Integrations	Integration
9	Module 9 — قواعد البيانات	Integration
10	Module 10 — AI Automation	AI & Resilience
11	Module 11 — Error Handling	AI & Resilience
12	Module 12 — Security	AI & Resilience
13	Module 13 — Production Workflows	Production
14	Module 14 — Deployment	Production
15	Module 15 — Advanced Architecture	Production
—	كيف يفكر المحترف	—
—	Glossary	—
—	Final Checklist	—

0. كيف تستخدم هذا المنهج

0.1 العقد بينك وبين هذا المنهج

هذا ليس كتابًا تقرأه. هذا ورشة عمل مكتوبة.

ما يقدمه المنهج	ما يجب أن تقدمه أنت
شرح المفهوم	فتح n8n والبناء فعليًا
مثال مبني خطوة بخطوة	تنفيذه بيدك، لا قراءته
تحدي بلا حل مباشر	محاولة صادقة قبل النظر للحل
أخطاء شائعة	كسر ال workflow عمدًا وإصلاحه
معياري إتمام	الصدق مع نفسك في تطبيقه

0.2 رموز المنهج

الرمز	المعنى
💡	مفهوم أساسي
⚠️	تحذير — خطأ شائع أو سلوك غير متوقع
🧠	فكر كمهندس — طريقة تفكير لا معلومة
🔧	تطبيق عملي (Build Along)
🦾	تحدي — تحله وحدك
🚀	تطبيق في العالم الحقيقي
🔒	أمان
🐛	تشخيص وإصلاح
✓	معياري إتمام
📌	تحقق من الإصدار

0.3 القاعدة الذهبية

🧠 لا تنسخ. افهم ثم ابن.

نسخ workflow يعطيك نتيجة اليوم. فهم *لماذا* بُني هكذا يعطيك القدرة على بناء ألف workflow لاحقًا.
الاختبار: أغلق المنهج، وابن نفس الشيء من الذاكرة. إن لم تستطع، لم تتعلمه بعد.

المستوى 1 — Foundations

Module 1 — مقدمة في الأتمتة

الهدف: أن تفهم متى تُستخدم الأتمتة ومتى لا تُستخدم — قبل أن تلمس أي أداة. المدة: 60-90 دقيقة · المتطلبات: لا شيء

المهارات المكتسبة

- تمييز العمليات القابلة للأتمتة من غير القابلة
- حساب العائد على الاستثمار (ROI) لأي أتمتة قبل بنائها
- اختيار n8n أو رفضه بناءً على معايير موضوعية
- التفكير بمنطق "المدخلات ← المعالجة ← المخرجات"

1.1 ما هي الأتمتة؟

💡 الفكرة ببساطة

الأتمتة هي: "إذا حدث X، افعل Y — دون أن أكون حاضراً."

مثال من حياتك: عندما يصلك إشعار على هاتفك بأن حوالة وصلت لحسابك — لم يجلس موظف بنك ليكتب لك رسالة. هناك نظام يراقب الحساب، وعند حدوث إيداع يرسل إشعاراً. هذه أتمتة.

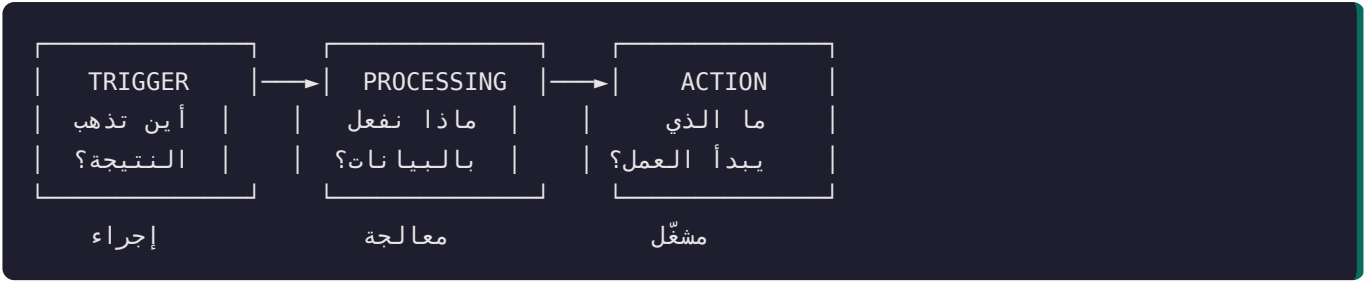
؟ لماذا يهم؟

لأن معظم الناس يخطئون في تعريف الأتمتة فيبنون الشيء الخطأ. الأتمتة ليست "برنامج ذكي". هي نقل قرار متكرر ومحدد من رأس إنسان إلى نظام. القاعدة: إن لم تستطع كتابة القرار على شكل قواعد واضحة، فأنت لا تحتاج أتمتة — تحتاج أولاً أن توضّح العملية.

🧠 فُكّر كمهندس: الأتمتة السيئة تُسرّع الفوضى. إن كانت عمليتك اليدوية فوضوية، فأتمتتها تنتج فوضى أسرع وأصعب في التنج. رتب العملية أولاً، ثم أتمتها.

⚙️ كيف تعمل؟ — البنية الكونية لأي أتمتة

كل أتمتة في العالم، مهما بلغ تعقيدها، تتكون من ثلاث طبقات:



الطبقة	السؤال	أمثلة
Trigger	ما الحدث الذي يبدأ كل شيء؟	وصل بريد · امتلأ نموذج · حان وقت محدد · نادى نظام آخر
Processing	ما التحويل المطلوب؟	تصفية · تحويل شكل · حساب · قرار · استدعاء AI
Action	أين تُودع النتيجة؟	حفظ في قاعدة بيانات · إرسال رسالة · تحديث سجل

🔗 إن لم تستطع تسمية الطبقات الثلاث لفكرتك، فأنت لست جاهزًا للبناء بعد. هذا الاختبار البسيط سيوفر عليك ساعات ضائعة طوال حياتك المهنية.

🔧 تطبيق ذهني (بلا حاسوب)

خذ ورقة. اكتب ثلاث عمليات متكررة تقوم بها أسبوعيًا. لكل واحدة، املأ:

العملية	Trigger	Processing	Action
مثال: متابعة العملاء الجدد	وصول عميل جديد من النموذج	تحديد أولويته حسب الميزانية	إضافته للجدول + إشعار الفريق
...عمليتك 1			
...عمليتك 2			
...عمليتك 3			

هذه الورقة هي أول وثيقة معمارية تكتبها. احتفظ بها.

1.2 متى تستحق العملية الأتمتة؟ (معادلة ROI)

؟ لماذا يهم؟

أكبر مخرج في مجال الأتمتة ليس الكود السيئ — بل أتمتة أشياء لا تستحق الأتمتة. مبتدئون يقضون 12 ساعة لأتمتة مهمة تستغرق دقيقتين شهريًا.

الوقت الموقّر سنويًا = (الدقائق لكل مرة) × (عدد المرات سنويًا)
نقطة التعادل = وقت البناء + (وقت الصيانة سنويًا)

استحققت الأتمتة إذا: الوقت الموقّر < نقطة التعادل × 3

⚠️ لا تنس وقت الصيانة. أشهر خطأ في حساب ROI. الـ APIs تتغير، والصلاحيات تنتهي، والخدمات تتوقف. قدر 10-20% من وقت البناء كصيانة سنوية. الأنظمة التي تعتمد على AI أو خدمات خارجية كثيرة تكون في الطرف الأعلى.

مثال حقيقي 🚀

البند	القيمة
المهمة	نسخ بيانات عميل جديد من البريد إلى الجدول
الوقت لكل مرة	4 دقائق
التكرار	20 مرة أسبوعيًا = 1040 سنويًا
الوقت الموقّر	4160 دقيقة ≈ 69 ساعة سنويًا
وقت البناء	5 ساعات
الصيانة السنوية	~1 ساعة
نقطة التعادل	6 ساعات
الحكم	69 > 18 ✓ تستحق بقوة

Challenge 1.2 🏆

احسب ROI لأكثر عملية متكررة في عملك. إن كانت النتيجة سلبية — لا تؤتمتها، واختر غيرها. القدرة على قول "هذا لا يستحق" مهارة محترفين.

🚀 للعمل الحر: هذه المعادلة هي أداة بيعك الأقوى. العميل لا يشتري "workflow" — يشتري 69 ساعة من وقت موظفيه. حوّل دائمًا الحديث التقني إلى ساعات ونقود.

1.3 ما هو n8n؟

الفكرة ببساطة 💡

n8n أداة تتيح لك رسم الأتمتة بدل كتابتها كودًا: تضع صناديق (Nodes) وتصل بينها بخطوط، فتنتقل البيانات من صندوق لآخر وتتغير في الطريق. الاسم يُنطق: "n-eight-n" (اختصار لـ nodemation).

الخاصية	ماذا تعني عمليًا
Fair-code / مفتوح المصدر	تستطيع تحميله وتشغيله على خادمك — بياناتك لا تغادر بنيتك
Self-hostable	لا تدفع لكل عملية تنفيذ إن استضافته بنفسك
Code when needed	واجهة مرئية، لكن يمكنك كتابة JavaScript/Python عند الحاجة
+400 عقدة جاهزة	تكاملات مبنية مسبقًا مع الخدمات الشهيرة
HTTP Request عام	أهم ميزة: يصل لأي API في العالم حتى لو لم تكن له عقدة
AI-native	عقد مخصصة لـ LLMs و AI Agents و Vector Stores

🧠 أهم سطر في هذا الجدول هو "HTTP Request". المبتدئ يسأل: "هل يوجد Node لخدمة X؟" المحترف يسأل: "هل لخدمة X واجهة API؟" — وإن كان الجواب نعم، فهي مدعومة. هذا الفرق وحده يوسع قدرتك من 400 خدمة إلى كل خدمة على الإنترنت.

📌 تنبيه إصدار

n8n يتطور سريعًا. المنهج مُتحقق مقابل 2.37.10.

الإصدار	الحالة (سبتمبر 2026)
x.2	✓ المستقر الحالي — المنهج مبني عليه
x.1	⚠️ ما زال يُصان، ومعظم المحتوى القديم على الإنترنت مبني عليه
3.0	📋 تغييرات كاسرة موثقة — راجعها قبل الترقية

كل عقدة لها **typeVersion**. عندما لا يعمل حل وجدته على الإنترنت، أول احتمال هو اختلاف الإصدار — لا قصور فيك.

1.4 متى تستخدم n8n ومتى لا تستخدمه؟

هذا القسم يمنعك من إهدار أسابيع في الأداة الخطأ.

الحالة	لماذا
تربط عدة خدمات ببعضها	هذا جوهر تخصصه
تحتاج منطقًا شرطيًا وتفرعات	IF/Switch/Merge مصممة لهذا
تريد رؤية التدفق بصريًا	التمحيص أسرع بمراحل
البيانات حساسة ويجب ألا تغادر خادمك	الاستضافة الذاتية
تبني AI Agents بأدوات وذاكرة	دعم أصيل ومتقدم
المنطق يتغير كثيرًا	التعديل المرئي أسرع من إعادة النشر
تريد تسليم نظام لعمل غير تقني	يستطيع رؤية النظام وفهمه

لا تستخدم n8n عندما

الحالة	البديل الأصح
معالجة ملايين السجلات في الثانية	Apache Kafka / Spark
تحتاج زمن استجابة أقل من 50ms	خدمة مخصصة (Go/Rust)
المنطق خوارزمي معقد جدًا	كود عادي في مستودع + اختبارات
تطبيق كامل يواجه مستخدم غنية	إطار ويب حقيقي
مهمة واحدة بسيطة لمرة واحدة	سكريبت من 10 أسطر
فريقك مهندسو برمجيات يفضلون الكود	كود + CI/CD

⚠️ المصيدة الأشهر: استخدام n8n كقاعدة بيانات أو كخادم تطبيقات كامل. n8n منسق (Orchestrator) — يحرك البيانات ويقرر ويستدعي. ليس مكانًا لتخزين حالة التطبيق طويلة الأمد، ولا لخدمة آلاف الطلبات في الثانية.

مقياس الحكم في جملة واحدة

إذا كانت المشكلة "ربط أشياء + اتخاذ قرارات" → n8n ممتاز. إذا كانت "حساب ثقل أو أداء عالٍ" → n8n ليس الأداة.

1.5 الفرق بين Automation و AI Automation

💡 الفكرة ببساطة

البُعد	Automation تقليدية	AI Automation
القرار	قواعد كتبها أنت	نموذج يستنتج
المدخلات	منظمة (حقول محددة)	غير منظمة (نص حر، صور، صوت)
المخرجات	حتمية — نفس المدخل = نفس المخرج	احتمالية — قد تختلف
الخطأ	يظهر كرسالة خطأ	قد يظهر كإجابة خاطئة تبدو صحيحة
التكلفة	شبه صفر لكل تنفيذ	تكلفة لكل استدعاء
السرعة	ميلي ثانية	ثواني

⚠️ التحذير الأهم في هذا المنهج

الأتمتة التقليدية تفشل بصوت عالٍ. الأتمتة بالذكاء الاصطناعي تفشل بصمت. عندما ينكسر HTTP Request، ترى خطأ أحمر واضحًا. عندما يهلوس LLM ويصنف عميلًا غاضبًا على أنه "مهتم"، لا يظهر أي خطأ — النظام يواصل بثقة تامة وهو مخطئ.

القاعدة العملية:

هل يمكن التعبير عن القرار بقواعد واضحة؟

- (أرخص، أسرع، أوثق، قابل للاختبار) IF / Switch نعم — استخدم
- أضعف إلزاميًا + AI لا — استخدم
 - (مخرج بصيغة محددة) Structured Output
 - (Validation) التحقق من صحة المخرج
 - (Fallback) مسار احتياطي عند فشل التحليل
 - (Human-in-the-loop) تدخل بشري للحالات الحرجة

🚀 في مشاريع العملاء: أكثر خطأ مكلف هو استخدام AI حيث تكفي قاعدة IF. "صنف الميزانية: أكثر من 50,000 = عميل مؤهل" — هذه لا تحتاج نموذج لغوي. تحتاج IF. استخدم AI حين تكون المدخلات نصًا حرًا غير متوقع: رسالة عميل، وصف مشكلة، سيرة ذاتية.

1.6 كيف يفكر مهندس الأتمتة؟

الفروق الجوهرية

المبتدئ	المحترف
"أي Node أستخدم؟"	"ما شكل البيانات في كل مرحلة؟"
يبني ثم يفكر بالأخطاء	يصمم الأخطاء أولاً
يختبر المسار الناجح فقط	يختبر الفشل عمداً
workflow واحد عملاق	وحدات صغيرة قابلة لإعادة الاستخدام
"اشتغل!"	"هل يشتغل بعد 6 أشهر ومع 10 أضعاف البيانات؟"
يسأل AI عند أول خطأ	يشخص أولاً، ثم يسأل بسؤال دقيق
يخزن السر في ال workflow	يستخدم Credentials
يفترض نجاح API دائماً	يفترض الفشل ويخطط له

الأسئلة الاثنا عشر — قبل بناء أي workflow

اطبع هذه القائمة وضعها أمامك. الإجابة عليها قبل البناء توفر أضعاف وقتها:

1. ما الحدث الذي يبدأ العملية بالضبط؟
2. ما شكل البيانات الداخلة؟ (اطلب عينة حقيقية)
3. ما شكل المخرج المطلوب بالضبط؟
4. ماذا يحدث إذا كانت البيانات ناقصة؟
5. ماذا يحدث إذا فشل ال API الخارجي؟
6. ماذا يحدث إذا وصل الحدث مرتين؟
7. ما الحجم المتوقع؟ (10 يومياً أم 10,000؟)
8. أين تُخزن البيانات، ومن يملك الوصول؟
9. كيف أحمي الأسرار (المفاتيح والصلاحيات)؟
10. كيف أعرف أن النظام توقف عن العمل؟
11. كيف أختبره دون التأثير على بيانات حقيقية؟
12. هل يمكن تبسيط المعمارية؟

🔑 السؤالان 6 و 10 هما ما يميز المحترف. المبتدئ لا يفكر فيهما أبداً، وهما سبب معظم كوارث الإنتاج.

Common Mistakes — Module 1 ⚠️

الخطأ	لماذا يحدث	التصحيح
البدء بالأداة لا بالمشكلة	حماس تقني	اكتب Trigger/Processing/Action أولاً
أتمتة عملية فوضوية	ظن أن الأتمتة ترتب	رتب العملية يدويًا أولاً
استخدام AI حيث تكفي القاعدة	انبهار بالتقنية	استخدم IF ما لم يكن المدخل نصًا حرًا
تجاهل وقت الصيانة	تفاؤل	أضف 10-20% لحساب ROI
افتراض دعم كل الخدمات جاهزًا	عدم معرفة HTTP Request	كل خدمة لها API = مدعومة

Best Practices — Module 1 ★

1. اكتب المشكلة بجملة واحدة قبل فتح n8n.
2. احسب ROI. إن كان سلبياً، لا تب.
3. ارسم الطبقات الثلاث على ورق.
4. أسأل: "ماذا لو فشل هذا؟" عند كل خطوة.
5. ابدأ بأصغر نسخة تعمل، ثم وسع.

Mini Quiz — Module 1 📝

1. اذكر الطبقات الثلاث لأي أتمتة، مع مثال لكل طبقة.
2. عملية تستغرق 3 دقائق، تتكرر 5 مرات أسبوعياً. بناؤها 8 ساعات وصيانتها ساعتان سنوياً. هل تستحق؟ اعرض الحساب.
3. متى تختار IF بدل AI؟ أعط مثالاً واقعياً لكل حالة.
4. لماذا يُقال إن "الأتمتة بالذكاء الاصطناعي تفشل بصمت"؟
5. خدمة ليس لها Node في n8n. هل يمكن ربطها؟ كيف؟

الإجابات ▼

1. **Trigger** (وصول بريد) → **Processing** (استخراج المرفق وقراءته) → **Action** (حفظ في Drive وإشعار).
2. الموقر $3 \times 5 \times 52 = 780$ دقيقة = 13 ساعة/سنة. التعادل $8 + 2 = 10$ ساعات. الشرط: $13 < 30$ ؟ لا ← لا تستحق حالياً. (تستحق فقط إن زاد التكرار أو قل وقت البناء).
3. IF عندما يكون القرار قاعدة صريحة على بيانات منظّمة ("الميزانية < 50,000"). AI عندما يكون المدخل نصاً حرًا ("العميل كتب: أبغى أعرف الأسعار بس مو مستعجل").
4. لأن النموذج يُنتج مخرَجًا يبدو سليماً شكلاً حتى حين يكون خاطئاً مضموناً؛ لا يرفع استثناءً، فيمرّ الخطأ في النظام دون إنذار.
5. نعم — عبر **HTTP Request Node**. بشرط أن يكون للخدمة API. عقد الخدمات الجاهزة اختصار للراحة لا شرط للربط.

Definition of Done — Module 1 ✓

- ☐ أستطيع تفكيك أي فكرة أتمتة إلى Trigger / Processing / Action.
- ☐ حسبت ROI لعملية حقيقية من عملي.
- ☐ أستطيع تبرير اختيار n8n أو رفضه بمعايير موضوعية.
- ☐ أعرف متى أستخدم AI ومتى أستخدم قاعدة بسيطة.
- ☐ كتبت الأسئلة الاثني عشر في مكان أرجع إليه.

Module 2 — واجهة n8n ونموذج التنفيذ

الهدف: أن تفهم كيف تتحرك البيانات فعليًا داخل n8n — وهو الفهم الذي بدونهُ يستحيل التصحيح. المدة: 90-120 دقيقة · المتطلبات: Module 1

المهارات المكتسبة

- قراءة أي workflow وفهم تدفق بياناته
- استخدام لوحة Executions للتشخيص
- التمييز الحاسم بين التنفيذ اليدوي والإنتاجي
- إدارة Credentials بشكل آمن

2.1 المفاهيم الأساسية

الخريطة الذهنية

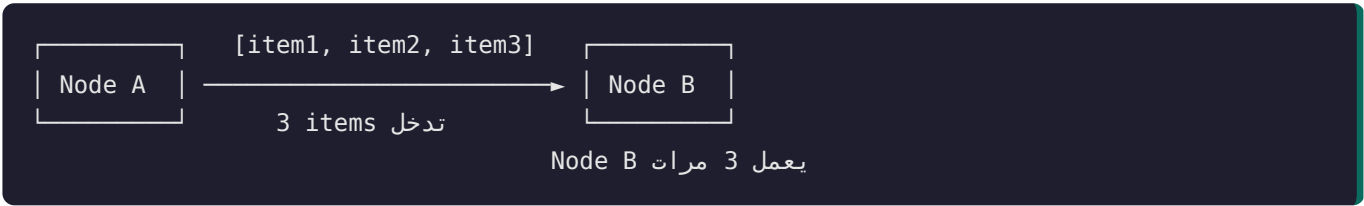
Workflow — مجموعة عقد متصلة تنفذ عملية واحدة كاملة

- Node — صندوق يفعل شيئًا واحدًا
 - Trigger — العقدة التي تبدأ التنفيذ (واحدة فعّالة لكل مسار)
 - Action — عقدة تعمل على البيانات القادمة
- Connection — الخط الذي تسير عليه البيانات
- Item — وحدة البيانات الواحدة (سجل واحد)
- Execution — workflow مرة واحدة اشتغل فيها الـ
- Credential — بيانات الدخول لخدمة خارجية (مخزنة مشفرة ومنفصلة)

🔔 n8n لا ينقل "بيانات". n8n ينقل قائمة من الـ items.

هذه الجملة هي مفتاح فهم 80% من سلوك n8n الذي يحير المبتدئين.
كل عقدة:

- 1. تستقبل مصفوفة من الـ items (حتى لو كان item واحدًا فهي مصفوفة من عنصر واحد).
- 2. تُنفَّذ مرة لكل item (في معظم العقد).
- 3. تُخرج مصفوفة من الـ items.



⚠️ النتائج العملية لهذه القاعدة

السلوك	التفسير
أرسلت بريدًا فوصلت 50 رسالة	دخلت 50 item، فعملت العقدة 50 مرة
العقدة التالية لم تعمل إطلاقًا	العقدة السابقة أخرجت 0 items — ليس خطأ، بل لا شيء يُعالج
الاستعلام نُفِّذ مرات كثيرة	كل item يستدعي تنفيذًا منفصلًا

🔔 قاعدة تشخيص ذهبية: عندما لا يعمل شيء كما تتوقع، أول سؤال دائمًا: "كم item يدخل هذه العقدة؟" افتح العقدة وانظر لعدد الـ items في لوحة الإدخال. أكثر من نصف المشاكل تُحل عند هذا السؤال وحده.

2.2 لوحة Executions — أدواتك الأولى في التشخيص

؟ لماذا يهم؟

هذه اللوحة هي الفرق بين مشجّص ومخجّن. من يسأل AI عند كل خطأ عادةً لم يتعلم قراءتها.

العمود	المعنى	استخدامه في التشخيص
Status	Success / Error / Running / Waiting	التصفية على Error للعثور على المشاكل
Started	وقت البدء	ربط الخطأ بحدث خارجي
Run Time	مدة التنفيذ	كشف البطء والاختناقات
Mode	كيف بدأ (Manual / Trigger / Webhook / Retry)	حاسم — انظر §2.3
Execution ID	معرف فريد	للرجوع والمشاركة

Build Along 🛠️ — قراءة تنفيذ فاشل

1. افتح Executions من القائمة الجانبية.
 2. صِف على Error.
 3. افتح أي تنفيذ فاشل.
 4. العقدة الحمراء هي مكان الفشل.
 5. اضغط عليها وافحص بالترتيب:
- Input ← ما الذي دخل فعلاً؟ (غالبًا هنا يكمن السبب)
 - Output / Error ← ما رسالة الخطأ الحرفية؟
6. اسأل: هل المشكلة في البيانات الداخلة أم في إعداد العقدة؟

🧐 بروتوكول التشخيص المكوّن من 4 أسئلة — احفظه:

1. أين فشل؟ (أي عقدة بالضبط)
 2. ما الذي دخلها؟ (افحص Input، لا تفترض)
 3. ما الذي توقعته العقدة؟ (راجع إعداداتها)
 4. أين الفجوة بين 2 و 3؟ ← هذه هي المشكلة، دائمًا.
- هذا البروتوكول يحل أكثر من 80% من الأخطاء دون أي بحث خارجي.

2.3 التنفيذ اليدوي مقابل الإنتاجي — الفرق الذي يوقع الجميع

⚠️ أخطر سوء فهم للمبتدئين

الضغط على "Execute Workflow" لا يعني أن الـ workflow يعمل.

البُعد	Manual Execution	Production Execution
كيف يبدأ	ضغط زر منك	ال Trigger الحقيقي
هل يحتاج تفعيل؟	لا	نعم — يجب أن يكون Active
Webhook URL	رابط Test مؤقت	رابط Production ثابت
الاستماع لـ Webhook	مرة واحدة فقط	دائم
Schedule Trigger	لا يعمل إطلاقاً	يعمل حسب الجدول

السيناريو الكلاسيكي

بنيت workflow يعمل بالضغط اليدوي. أغلقت المتصفح. عدت مبأخاً فلم يحدث شيء. السبب: ال workflow غير مُفعّل (Active = OFF).
الجدولة الزمنية لا تعمل إلا على workflow مفعّل.

قائمة التحقق قبل قول "لا يعمل":

- ☐ هل مفتاح **Active** مفعّل (أعلى يمين الشاشة)؟
- ☐ هل أستخدم رابط **Production** لا رابط **Test**؟
- ☐ هل حفظت التغييرات بعد آخر تعديل؟
- ☐ هل ظهر التنفيذ في لوحة Executions أصلاً؟ (إن لم يظهر، المشكلة في ال Trigger لا في المنطق)

2.4 Credentials — إدارة الأسرار

الفكرة ببساطة

ال Credential = بطاقة دخولك لخدمة خارجية، محفوظة في خزانة منفصلة عن ال workflow.

؟ لماذا يهم؟ (سبب أمني، لا تنظيمي)

بدونها ✕	مع Credentials ✔
مكتوبة نصاً صريحاً داخل العقدة	مخزّنة مشفرة
تُصدّر مع الملف وتُسَرَّب	لا تظهر عند تصدير ال workflow
تُعدّل في كل عقدة على حدة	تُحدّث في مكان واحد
لا	قابلة لمشاركة الاستخدام دون كشف القيمة

🔒 القاعدة المطلقة: لا تكتب مفتاح API أو كلمة مرور داخل حقل في عقدة. أبدًا، ولا حتى "مؤقتًا للتجربة".

السبب: التصدير والمشاركة والنسخ الاحتياطي كلها تحمل الملف بما فيه. المفتاح "المؤقت" يبقى في تاريخ Git للأبد. التفاصيل الكاملة في **Module** 12.

Build Along 🛠️

1. **Credentials → Add Credential**.

2. اختر النوع (مثلًا Google Sheets OAuth2).

3. أكمل التفويض.

4. سيها بوضوح: حساب العمل - Google Sheets لا credential 1.

★ تسمية احترافية: عندما تدير 5 عملاء، عميل أ (إنتاج) - Postgres تمنع كارثة الكتابة على قاعدة بيانات العميل الخطأ.

2.5 Pinned Data — الاختبار دون استهلاك

💡 الفكرة

Pin Data يُثبت مخرجات عقدة، فُستخدم القيمة المثبتة في كل تنفيذ يدوي لاحق بدل استدعاء الخدمة فعليًا.

? لماذا يهم؟ — ثلاثة أسباب عملية

1. توفير المال: لا تستدعي API مدفوعًا (خاصة LLM) عند كل تجربة.

2. توفير الوقت: لا تنتظر webhook حقيقيًا في كل مرة.

3. الثبات: تختبر منطقك على نفس البيانات تمامًا، فتعزل المتغيرات.

🛠️ الاستخدام

نقذ العقدة مرة → افتح مخرجاتها → اضغط أيقونة **Pin** 📌 → ابن بقية ال workflow على البيانات المثبتة.

⚠️ البيانات المثبتة تُستخدم في التنفيذ اليدوي فقط. الإنتاج يستخدم البيانات الحقيقية دائمًا. ⚠️ لكن: انزع التثبيت قبل التسليم النهائي — تثبيت منسي على عقدة يجعلك تختبر وهمًا وتظن أن كل شيء سليم.

Common Mistakes — Module 2 ⚠️

الخطأ	العَرَض	التصحيح
نسيان التفعيل	"لا يعمل تلقائيًا"	فعل Active
استخدام رابط Test في الإنتاج	يعمل مرة ثم يتوقف	استخدم Production URL
تجاهل عدد الـ items	تكرار غير متوقع أو لا شيء	افحص عداد Input
كتابة المفتاح في العقدة	تسريب عند التصدير	استخدم Credential
نسيان نزع Pin	يعمل عندك ويفشل إنتاجيًا	انزع التثبيت قبل التسليم
عدم فتح Executions	تخمين بلا دليل	ابدأ التشخيص منها دائمًا

Best Practices — Module 2 ★

1. سيم كل عقدة بما تفعله: التحقق من البريد لا IF1 . نظام بعشرين عقدة مجهولة الاسم غير قابل للصيانة.
2. أضيف Sticky Notes لشرح القرارات غير البديهية — أنت بعد ستة أشهر شخص آخر.
3. اكتب وصفًا للـ workflow يشرح: ماذا يفعل، ما مدخلاته، ما مخرجاته.
4. افحص Executions يوميًا في الأنظمة الإنتاجية.
5. استخدم Pin Data عند التطوير مع APIs مدفوعة.

Mini Quiz — Module 2 🖋️

1. عقدة تستقبل 5 items وترسل بريدًا. كم رسالة تُرسل؟ ولماذا؟
2. ما الفرق بين Test URL و Production URL للـ Webhook؟
3. workflow مجدول لا يعمل رغم أنه يعمل يدويًا. ما أول شيء تفحصه؟
4. لماذا لا يجوز كتابة API Key داخل حقل في العقدة؟
5. اذكر أسئلة بروتوكول التشخيص الأربعة بالترتيب.

الإجابات ▼

1. 5 رسائل. العقدة تُنفذ مرة لكل item — هذا السلوك الافتراضي في n8n.
2. Test URL يستمع لطلب واحد فقط وأثناء وضع الاختبار؛ Production URL ثابت ودائم ويعمل فقط عندما يكون الـ workflow مفعلاً (Active).
3. مفتاح Active. الجدولة الزمنية لا تعمل على workflow غير مفعّل.
4. لأنه يُخزن نصًا صريحًا داخل تعريف الـ workflow، فيُصدّر ويُشارك ويُنسخ احتياطيًا معه — بينما الـ Credential يُخزن مشفرًا ومنفصلًا ولا يُصدّر.
5. (1) أين فشل؟ (2) ما الذي دخل؟ (3) ما الذي توقعته العقدة؟ (4) أين الفجوة بين 2 و 3؟

Definition of Done — Module 2 ✓

- ☐ • أستطيع شرح مفهوم ال items ولماذا تُنفَّذ العقدة عدة مرات.
- ☐ • أستخدم لوحة Executions لتشخيص فشل دون سؤال أحد.
- ☐ • أفَرِّق بين التنفيذ اليدوي والإنتاجي وأعرف متطلبات كل منهما.
- ☐ • أنشأت Credential واحدًا على الأقل بتسمية واضحة.
- ☐ • أستخدم Pin Data للاختبار.
- ☐ • أسمّي كل عقدة بما تفعله.

Module 3 — Triggers

الهدف: اختيار المشغل الصحيح لكل حالة — قرار يحدد معمارية النظام كله. المدة: 90 دقيقة · المتطلبات: Module 2

المهارات المكتسبة

- اختيار Trigger مناسب بمعايير واضحة
- التمييز بين Polling و Push وأثرهما على التكلفة والزمن
- استقبال بيانات خارجية عبر Webhook
- بناء نماذج تفاعلية بـ Form Trigger

3.1 التصنيف الجوهرى: Push مقابل Poll

💡 الفكرة ببساطة

النمط	التشبيه	كيف يعمل
Push	جرس الباب	الخدمة الخارجية تخبرك فور وقوع الحدث
Poll	النظر من النافذة كل دقيقة	أنت تسأل بشكل متكرر: "هل جدّ جديد؟"

المعيار	Push (Webhook)	Poll (Schedule)
زمن الاستجابة	فوري	حتى فترة الفحص
استهلاك الموارد	منخفض جدًا	مرتفع (تنفيذ حتى بلا بيانات)
التعقيد	يحتاج رابطًا عامًا + تأمينًا	أبسط
الموثوقية	قد تُفقد رسالة إن كان نظامك متوقفًا	يلتقط ما فات عند التشغيل التالي
الدعم	يتطلب دعم الخدمة له	يعمل مع أي API

🧠 قاعدة القرار: يدعم المزود webhook؟ → استخدمه. أسرع وأرخص. لا يدعم؟ → Polling، وأمل الفترة قدر ما يحتمله العمل.

الفحص كل دقيقة يعني 43,200 تنفيذًا شهريًا — معظمها فارغ. هذا يستهلك حصتك على n8n Cloud ويرهق خادمك على self-hosted. اسأل دائمًا: "هل يكفي كل 15 دقيقة؟" — الجواب عادةً نعم.

Manual Trigger 3.2

🔥 عقدة التطوير — لا تُستخدم في الإنتاج.

البند	التفصيل
ما هي	تبدأ التنفيذ بضغطة زر
متى	أثناء البناء والاختبار فقط
متى لا	أي شيء يجب أن يعمل تلقائيًا

⚠️ تسليم workflow للعميل بـ Manual Trigger = تسليم نظام لا يعمل. هذا خطأ يقع فيه المبتدئون فعليًا.

Schedule Trigger 3.3

🔥 n8n-nodes-base.scheduleTrigger — typeVersion 1.4

💡 الفكرة

ينفذ ال workflow على جدول زمني: كل ساعة، يوميًا 8 صباحًا، كل اثنين، أو بتعبير Cron.

حالات الاستخدام 🚀

- تقرير يومي صباحي
- مزامنة دورية بين نظامين
- تنظيف بيانات قديمة أسبوعيًا
- محرك متابعة (يفحص من يستحق تذكيرًا اليوم)

صيغة Cron ⚙️

```

— الدقيقة (0-59)
| — الساعة (0-23)
| | — اليوم من الشهر (1-31)
| | | — الشهر (1-12)
| | | | — اليوم من الأسبوع (0-6، 0=الأحد)
| | | | |
* * * * *
    
```

التعبير	المعنى
* * * * * 8 0	يوميًا الساعة 8:00
* * * * * */15	كل 15 دقيقة
0 9 * * 1	كل اثنين الساعة 9:00
0 0 1 * *	أول كل شهر منتصف الليل

Common Mistakes ⚠️

الخطأ	الأثر	التصحيح
المنطقة الزمنية	التقرير يصل 11 مساءً بدل 8 صباحًا	اضبط Timezone في إعدادات ال workflow لا الحساب فقط
فحص كثيف بلا داع	استهلاك الحصة	أطل الفترة لأقصى ما يحتمله العمل
نسيان التفعيل	لا يعمل إطلاقًا	Active = ON
تجاهل التنفيذات المتداخلة	تشغيلان معًا يفسدان البيانات	إن كان التنفيذ أطول من الفترة، أطل الفترة أو أضف قفلاً

🧠 سؤال المحترف: "ماذا لو كان n8n متوقفًا وقت الجدولة؟" الجواب: تُقوّت تلك الدورة. لذا لا تعتمد على الجدولة وحدها للعمليات الحرجة — صمّم ال workflow ليعالج "كل ما لم يُعالج بعد" بناءً على حالة في قاعدة البيانات، لا "ما حدث في آخر 24 ساعة". هذا الفرق يسمى التصميم القائم على الحالة مقابل القائم على الزمن، وهو أحد أهم فروق النضج الهندسي.

Webhook Trigger 3.4

typeVersion 2.1 — n8n-nodes-base.webhook 📌

الفكرة ببساطة 💡

ال Webhook يعطي workflow عنوان URL خاصًا. أي نظام يرسل طلبًا لهذا العنوان → يبدأ التنفيذ فورًا وتصل البيانات معه.

? لماذا يهم؟

هذه العقدة تحوّل n8n من "أداة أتمتة" إلى خادم API. بها تستطيع بناء نقاط نهاية (endpoints) تستقبل من أي نظام في العالم.

الإعدادات الحاسمة ⚙️

الإعداد	الخيارات	متى
HTTP Method	GET / POST / PUT / PATCH / DELETE	POST للأغلب (استقبال بيانات)
Path	مسار مخصص	اجعله غير قابل للتخمين
Authentication	None / Basic / Header	لا تتركه None في الإنتاج 🚫
Respond	Immediately / When Last Node Finishes / Using Respond to Webhook Node	حسب حاجتك للرد

Production URL مقابل Test URL ⚠️

Production URL	Test URL	
عندما يكون ال workflow Active	عند الضغط على "Listen for test event"	متى يعمل
غير محدود	واحد فقط ثم يتوقف	عدد الطلبات
الإنتاج	التطوير	الاستخدام

🐛 أشهر مشكلة webhook على الإطلاق: "اشتغل مرة واحدة ثم توقف". السبب: استخدام Test URL. الحل: فعّل ال workflow واستخدم Production URL.

تحذير أمني مبكر 🚫

Webhook بلا مصادقة = باب مفتوح للإنترنت كله.
أي شخص يعرف الرابط يستطيع تشغيل ال workflow بأي بيانات: إغراق قاعدة بياناتك، استنزاف رصيد ال AI لديك، إرسال رسائل باسمك.
الحد الأدنى المقبول: Header Auth بقيمة سرية طويلة. المعالجة الكاملة في Module 12.

1. أضف Webhook node.

2. Path = POST ، Method = test-lead .

3. Listen for test event اضغط.

4. انسخ Test URL وأرسل إليه طلبًا:

```
curl -X POST '<TEST_URL_HERE>' \
-H 'Content-Type: application/json' \
-d '{"name": "محمد", "email": "m@example.com", "budget": 75000}'
```

5. افحص المخرجات: أين وجدت البيانات؟ داخل body — وليس في جذر الـ item.

💡 هذه ملاحظة مهمة: الـ Webhook يعطيك headers و params و query و body . بياناتك الفعلية في body . للوصول إليها: `{{ $json.body.name }}` — لا `{{ $json.name }}` . هذا مصدر إحباط شائع جدًا للمبتدئين.

Challenge 3.4 🏆

استقبل webhook يحمل `{ "order_id": 123, "items": [...], "total": 450 }` ، وأخرج item واحدًا يحتوي فقط على `order_id` و `total` . لا تنظر للحل. ستحتاج Module 5 و 6 — إن لم تستطع الآن، غُد بعدهما. تسجيل عجزك الآن جزء من التعلم.

Form Trigger 3.5

`n8n-nodes-base.formTrigger` — `typeVersion 2.6` · العقدة المكتملة `n8n-nodes-base.form` (v2.5) لصفحات إضافية 📌

الفكرة 💡

ينشئ n8n نموذجًا ويب حقيقيًا برابط، وعند الإرسال تبدأ الأتمتة ببيانات النموذج.

لماذا هو ذهبي للعمل الحر؟ 🚀

لا تحتاج موقعًا ولا مطور واجهات. تبني نموذج التقاط عملاء وتسلمه للعميل في نصف ساعة. مع عقدة Form تضيف صفحات متعددة و صفحة نهاية مخصصة.

الاستخدام	مثال
التقاط العملاء	نموذج "اطلب عرض سعر"
طلبات داخلية	طلب إجازة → موافقة
استبيانات	تقييم بعد الخدمة
مدخلات بشرية داخل عملية	مراجعة واعتماد

App / Event Triggers 3.6

عقد مشغلة مخصصة للخدمات: Postgres Trigger (v1)، WhatsApp Trigger (v1.5)، Telegram Trigger، Gmail Trigger، وغيرها.

🔴 ملاحظة مُتحققة حول WhatsApp Trigger: التحقق من الـ webhook تلقائي بالكامل. لا يوجد حقل "verify token" يضعه

المستخدم. عند التفعيل، يسجل n8n اشتراك Meta ويتحقق من التحدي مقابل معرف العقدة نفسه. إن طلب منك ملء خانة "Verify token" يدويًا في لوحة Meta، فالقيمة هي معرف العقدة، لا قيمة تخترعها. هذه معلومة تُهدر ساعات لمن لا يعرفها، وكثير من الشروحات القديمة تقول عكسها.

3.7 مصفوفة اختيار الـ Trigger

الحاجة	الـ Trigger
أختبر أثناء البناء	Manual
كل فترة زمنية	Schedule
نظام آخر يخبرني فور الحدث	Webhook
أجمع مدخلات من بشر	Form
حدث في خدمة لها عقدة مشغلة	App Trigger
workflow آخر يستدعيني	Execute Workflow Trigger (v1.2)
workflow آخر فشل	Error Trigger

Common Mistakes — Module 3 ⚠️

الخطأ	التصحيح
Manual Trigger في الإنتاج	استبدله بمشغل حقيقي
Test URL في الإنتاج	فعل واستخدم Production URL
Polling كثيف بلا داع	أطل الفترة
تجاهل المنطقة الزمنية	اضبط Timezone في الـ workflow
Webhook بلا مصادقة	Header Auth كحد أدنى
البحث عن البيانات في جذر الـ item	البيانات في <code>\$json.body</code>
اختراع verify token لواتساب	التحقق تلقائي

★ Best Practices — Module 3

1. فضل Webhook على Polling كلما أمكن.
2. اجعل مسار الـ webhook غير قابل للتخمين.
3. اضبط المنطقة الزمنية صراحةً.
4. صمم بمنطق الحالة لا الزمن للعمليات الحرجة.
5. وثق في وصف الـ workflow: من يستدعيه وبأي شكل بيانات.

🖋️ Mini Quiz — Module 3

1. متى تختار Polling رغم توفر Webhook؟
2. Webhook عمل مرة ثم توقف. لماذا؟
3. ما عيب * * * * * ؟
4. أين تجد بيانات الـ POST داخل مخرجات Webhook؟
5. تقرير يومي مطلوب 8 صباحًا لكن يصل 11 مساءً. ما السبب؟

▼ الإجابات

1. عندما تكون الموثوقية أهم من الفورية وتحتاج ضمان عدم فقدان أحداث وقعت أثناء توقف نظامك — الـ Polling يلتقط المتراكم عند التشغيل التالي، بينما الـ webhook أرسل وقت التوقف يضيع (ما لم يعد المزود المحاولة).
2. استخدم **Test URL**، وهو يستمع لطلب واحد فقط. الحل: تفعيل الـ workflow واستخدام **Production URL**.
3. ينفذ كل دقيقة = **43,200** تنفيذًا شهريًا، معظمها بلا بيانات — استهلاك حصة وموارد بلا مقابل، وقد يتداخل تنفيذان.
4. في `$json.body` — مثل `{{ $json.body.email }}`.
5. المنطقة الزمنية. الجدولة تعمل بتوقيت مختلف عن توقيتك المحلي — اضبط **Timezone** في إعدادات الـ workflow.

🖋️ Definition of Done — Module 3

- ☐ أختار الـ Trigger بمعايير لا بالعادة.
- ☐ بنيت **Schedule Trigger** يعمل بمنطقة زمنية صحيحة.
- ☐ بنيت Webhook واستقبلت به بيانات حقيقية عبر **curl**.
- ☐ أعرف الفرق بين **Test** و **Production URL** عمليًا.
- ☐ أدرك أن Webhook بلا مصادقة ثغرة أمنية.
- ☐ أعرف أن بيانات الـ POST في **body**.

المستوى 2 — Data Fluency

🧠 هذا المستوى هو العمود الفقري للمنهج كله. من يتقن الوحدات 4-6 يستطيع تعلّم أي شيء آخر في n8n وحده. ومن يتخطاها يبقى أسير النسخ واللصق للأبد — يبني ما رآه فقط، ويعجز عن بناء ما لم يره. إن كنت تنسخ الكود من AI ولا تفهمه، فهذه هي الوحدات التي ستغيّر ذلك. لا تتعجلها.

Module 4 — البيانات و JSON

الهدف: أن ترى البيانات بوضوح تام في أي نقطة من ال workflow — فلا تخمّن أبداً. المدة: 120 دقيقة · المتطلبات: Module 2

المهارات المكتسبة

- قراءة أي بنية JSON مهما تعقّدت
- التمييز الحاسم بين "عدة items" و "مصفوفة داخل item"
- الوصول لأي قيمة في بيانات متداخلة
- تحويل شكل البيانات بين المراحل

4.1 ما هو JSON؟

💡 الفكرة ببساطة

JSON هي لغة اتفقت عليها الأنظمة لتبادل البيانات. مثل الأرقام العربية: يفهمها الجميع رغم اختلاف اللغات.

كلمة JSON اختصار لـ JavaScript Object Notation. لكن لا تحتاج معرفة JavaScript لفهمها.

النوع	الشكل	مثال	ملاحظة حرجة
String	نص بين علامتي اقتباس	"محمد"	الأرقام بين علامتي اقتباس نصوص: "123"
Number	رقم بلا اقتباس	45000	45000 ≠ "45000"
Boolean	true أو false	true	صغيرة، بلا اقتباس
Null	null	null	"موجود لكن فارغ" — يختلف عن "غير موجود"
Object	{ }	{"name": "محمد"}	مجموعة أزواج مفتاح/قيمة
Array	[]	["أ", "ب"]	قائمة مرتبة

⚠️ الفرق بين "45000" و 45000 يسبب أخطاء حقيقية. 50000 > "45000" قد يعطي نتيجة غير متوقعة لأن المقارنة تجري على نص لا على رقم. عندما يفشل شرط IF بلا سبب ظاهر — افحص النوع أولاً. هذا من أشهر أسباب "الشرط لا يعمل وأنا متأكد أنه صحيح".

⚙️ البنية المتداخلة

```
{
  "customer": {
    "name": "محمد الحربي",
    "contact": { "email": "m@example.com", "phone": "+9665xxxxxxx" }
  },
  "order": {
    "id": 1042,
    "total": 450.50,
    "paid": true,
    "coupon": null,
    "items": [
      { "sku": "A-1", "qty": 2, "price": 150 },
      { "sku": "B-7", "qty": 1, "price": 150.50 }
    ]
  }
}
```

اقرأها كشجرة، ومسار الوصول هو الطريق من الجذر للورقة:

المسار	القيمة
customer.name	"محمد الحربي"
customer.contact.email	"m@example.com"
order.id	1042
order.items[0].sku	"A-1"
order.items[1].price	150.50
order.items.length	عدد المنتجات

💡 قاعدتان تحلان كل مشاكل المسارات:

- النقطة . للدخول إلى Object (بالاسم)
- الأقواس [n] للدخول إلى Array (بالرقم، يبدأ من 0)

Challenge 4.1 🏆

من نفس البنية أعلاه، اكتب مسار: (أ) رقم هاتف العميل (ب) كمية المنتج الثاني (ج) هل الطلب مدفوع.

📌 الحل ▼

(أ) customer.contact.phone (ب) order.items[1].qty — الثاني يعني الفهرس 1 ج) order.paid

4.2 المفهوم الأصعب: Items مقابل Arrays

⚠️ هذا القسم يحل أكثر من نصف حيرة المبتدئين في n8n. اقرأ مرتين.

منفصلة items الحالة أ - ثلاثة:

```
[
  { "name": "أحمد" },
  { "name": "سارة" },
  { "name": "خالد" }
]
```

→ العقدة التالية تعمل 3 مرات ✓

واحد يحوي مصفوفة item - الحالة ب:

```
[
  { "names": ["أحمد", "سارة", "خالد"] }
]
```

→ العقدة التالية تعمل مرة واحدة ⚠

نفس البيانات. سلوك مختلف تمامًا.

🚀 لماذا يهم فعليًا؟

API أعاد لك 50 منتجًا داخل item واحد:

```
[{ "products": [ {...}, {...}, ... 50 منتجًا ] }]
```

تريد إرسال بريد لكل منتج → أضفت عقدة البريد → وصلت رسالة واحدة فقط. السبب: هناك item واحد فقط. العقدة عملت مرة واحدة. الحل: حوّل المصفوفة إلى items منفصلة.

⚙ عقدة التحويل — احفظهما

العقدة	الاتجاه	typeVersion
Split Out	item فيه مصفوفة → عدة items	v1 n8n-nodes-base.splitOut
Aggregate	عدة item → items واحد فيه مصفوفة	v1 n8n-nodes-base.aggregate

```
[{products: [A,B,C]}] —Split Out→ [{A}, {B}, {C}]
[{A}, {B}, {C}] —Aggregate→ [{data: [A,B,C]}]
```

🔗 Aggregate تجمع items من فرع واحد. لجمع بيانات من فروع مختلفة استخدم Merge (شرحها في Module 6). الخلط بينهما شائع.

العَرَض	السبب المرجح	الحل
العقدة عملت مرة وتوقعت عدة مرات	item واحد فيه مصفوفة	Split Out
عملت 50 مرة وتوقعت مرة	item منفصلة	Aggregate، أو فَعَل Execute Once في إعدادات العقدة
لم تعمل إطلاقًا	items 0 داخلية	ارجع للعقدة السابقة — لماذا لم تُخرج شيئًا؟

Challenge 4.2 🤖

API أعاد:

```
[{ "status": "ok", "data": { "users": [ { "id":1,"n":"أ"}, { "id":2,"n":"ب"} ] } }]
```

تريد item منفصلًا لكل مستخدم. أي عقدة؟ وعلى أي حقل توجَّهها؟

الحل ✓ ▼

Split Out على الحقل `data.users`. النتيجة: `[{id:1,n:"أ"}, {id:2,n:"ب"}]` — والعقدة التالية ستعمل مرتين. **الفخ:** كتابة `users` فقط. المسار من جذر الـ `item`، ولا بد من المسار الكامل `data.users`.

4.3 التعامل مع البيانات المفقودة

سيناريو حقيقي ⚠️

```
{ "name": "محمد", "company": { "name": "شركة أ" } }
{ "name": "سارة" }
```

كتبْت `{{ $json.company.name }}` → نجح مع الأول، وانهار مع الثاني:
`TypeError: Cannot read property 'name' of undefined`

ثلاث طرق للحماية ⚙️

الطريقة	الصيغة	متى
Optional Chaining	<code>{{ \$json.company?.name }}</code>	تقبل قيمة فارغة
قيمة افتراضية	<code>{{ \$json.company?.name 'غير محدد' }}</code>	تريد بديلًا دائمًا
بوابة IF	فرع منفصل لناقصي البيانات	البيانات الناقصة تعني مسارًا مختلفًا

🔗 علامة الاستفهام ؟ تعني: "إن كان company غير موجود، أعط undefined بدل الانهيار." هذه العلامة وحدها ستمنع عنك عشرات الأخطاء، احفظها الآن.

🧠 قاعدة المحترف: كل حفل قادم من الخارج (API، webhook، إدخال بشري) يجب أن يُعامل كغائب محتمل. الفرق بين نظام هش ونظام صلب هو هذا الافتراض بالضبط. لا تثق ببيانات لم تولدها أنت.

Common Mistakes — Module 4 ⚠️

الخطأ	التصحيح
الخلط بين items ومصفوفة داخل item	Split Out / Aggregate
نسيان أن الفهرسة تبدأ من 0	العنصر الأول [0]
مقارنة نص برقم	حوّل النوع قبل المقارنة
افتراض وجود كل الحقول	استخدم ؟ وقيمًا افتراضية
مسار جزئي في Split Out	استخدم المسار الكامل من الجذر
الخلط بين null و undefined	null = موجود وفارغ، undefined = غير موجود

Mini Quiz — Module 4 🖋️

- الفرق بين `[[{a:1},{a:2}]]` و `[[{items:[1,2]}]]` في `JSON`؟
- من `{ "o": { "l": [ { "p": { "n": "س" } } ] } }` — مسار "س" ؟
- متى `Split Out` ومتى `Aggregate`؟
- لماذا `100 > 50` قد يعطي نتيجة غير متوقعة؟
- كيف تحمي `json.a.b.c` من الانهيار؟

📌 الإجابات ▼

- الأول `item`-ان، فتعمل العقدة التالية مرتين. الثاني `item` واحد فيه مصفوفة، فتعمل مرة واحدة.
- `o.l[0].p.n`
- `Split Out` لتفكيك مصفوفة داخل `item` إلى `Aggregate`. `items` لتجميع `items` في `item` واحد.
- لأن `"100"` نص لا رقم؛ المقارنة قد تتم كنص فيختلف الترتيب عن المتوقع رقميًا.
- `{ $json.a?.b?.c }` أو مع قيمة افتراضية: `{ 'افتراضي' || $json.a?.b?.c }`

Definition of Done — Module 4 ✓

- ☐ أقرأ أي JSON متداخل وأكتب مسار أي قيمة فيه.
- ☐ أفرّق فوراً بين "items" و "item" مصفوفة داخل item.
- ☐ أستخدم Split Out و Aggregate في موضعهما.
- ☐ أحمي كل وصول لبيانات خارجية من الغياب.
- ☐ أفحص أنواع البيانات قبل المقارنات.

Module 5 — Expressions

الهدف: أن تكتب أي تعبير ديناميكي بنفسك، دون نسخ. المدة: 120 دقيقة · المتطلبات: Module 4 (الزامي — لا تبدأ قبله)

المهارات المكتسبة

- الوصول لبيانات أي عقدة سابقة
- التعامل مع النصوص والتواريخ والمصفوفات
- كتابة شروط داخل التعابير
- تشخيص التعابير الفاشلة

5.1 الأساس

الفكرة 💡

القوسان المزدوجان `{{ }}` يقولان لـ `n8n`: "لا تأخذ ما بينهما نصاً — نفّذه."

```
نص ثابت:      مرحبا محمد
{{ $json.name }} تعبير:  مرحبا
(الحالي item حسب بيانات الـ) النتيجة:  مرحبا سارة
```

المتغير	المعنى	مثال
<code>\$json</code>	بيانات الـ item الحالي من العقدة السابقة مباشرة	<code>{{ \$json.email }}</code>
<code>\$node["اسم"].json</code>	بيانات عقدة محددة بالاسم	<code>{{ \$node["Webhook"].json.body.id }}</code>
<code>\$items()</code>	كل الـ items من عقدة	<code>{{ \$items("Set")[0].json.x }}</code>
<code>\$now</code>	اللحظة الحالية (كائن DateTime)	<code>{{ \$now.toISO() }}</code>
<code>\$today</code>	بداية اليوم	<code>{{ \$today.toISO() }}</code>
<code>\$execution.id</code>	معرف التنفيذ الحالي	للتتبع والسجلات
<code>\$workflow.name</code>	اسم الـ workflow	للسجلات
<code>\$runIndex</code>	رقم الدورة في الحلقات	داخل Loop
<code>\$vars</code>	متغيرات على مستوى النسخة	إعدادات مشتركة

⚠ **\$json** يشير للعقدة السابقة مباشرة فقط، إن مررت ببضع عقد وتحتاج بيانات من عقدة قديمة، لن يعمل **\$json** — استخدم `$node["اسم العقدة"].json`.

وهنا تظهر أهمية تسمية العقد: إن غيّرت اسم عقدة بعد الإشارة إليها، تنكسر كل التعبيرات التي تشير إليها. سمّ العقد جيدًا من البداية، وغيّر الأسماء مبكرًا لا متأخرًا.

5.2 النصوص (Strings)

العملية	التعبير	النتيجة
دمج	<code>{{ \$json.first + ' ' + \$json.last }}</code>	محمد الحربي
قالب	<code>{{ `مرحبا ` + \$json.name }}</code>	مرحبا محمد
أحرف كبيرة	<code>{{ \$json.code.toUpperCase() }}</code>	ABC
إزالة الفراغات	<code>{{ \$json.email.trim() }}</code>	بلا فراغات طرفية
استبدال	<code>{{ \$json.phone.replace(/\D/g, '') }}</code>	أرقام فقط
تقسيم	<code>{{ \$json.email.split('@')[1] }}</code>	example.com
احتواء	<code>{{ \$json.msg.includes('سعر') }}</code>	false / true
الطول	<code>{{ \$json.text.length }}</code>	عدد الأحرف
اقتطاع	<code>{{ \$json.text.slice(0, 100) }}</code>	أول 100 حرف

🚀 مثال إنتاجي — تنظيف رقم هاتف سعودي إلى صيغة دولية موحدة:

```
{{ '+966' + $json.phone.replace(/\D/g, '').replace(/^(966|0)/, '') }}
```

لماذا هذا مهم؟ أرقام الهواتف تصل بعشرات الصيغ: 0501234567 ، +966501234567 ، 966 50 123 4567 ، 050-123-4567 . بلا توحيد، سننشئ سجلات مكررة لنفس العميل في قاعدة بياناتك — وهذه مشكلة تكتشفها متأخراً وتكلف كثيراً في التنظيف. درس معماري: وُجد صيغة أي معزف (هاتف، بريد، رقم ضريبي) عند نقطة الدخول، لاحقاً.

5.3 التواريخ

n8n يستخدم Luxon للتواريخ.

العملية	التعبير
الآن بصيغة ISO	<code>{{ \$now.toISO() }}</code>
تنسيق مقروء	<code>{{ \$now.toFormat('yyyy-MM-dd HH:mm') }}</code>
بعد 3 أيام	<code>{{ \$now.plus({ days: 3 }).toISO() }}</code>
قبل أسبوع	<code>{{ \$now.minus({ weeks: 1 }).toISO() }}</code>
تحويل نص لتاريخ	<code>{{ DateTime.fromISO(\$json.date) }}</code>
الفرق بالأيام	<code>{{ Math.floor(\$now.diff(DateTime.fromISO(\$json.created), 'days').days) }}</code>
بداية اليوم	<code>{{ \$now.startOf('day').toISO() }}</code>

⚠️ **فخ المنطقة الزمنية:** `$now` يستخدم المنطقة الزمنية المضبوطة في إعدادات الـ `workflow`. إن كانت `UTC` وأنت في السعودية (`UTC+3`)، فد "اليوم" في نظامك يبدأ الساعة 3 فجراً بتوقيتك. الأثر: تقارير "أمس" تحتوي بيانات خاطئة، ومتابعات تُرسل في أوقات غريبة. **الحل:** اضبط `Timezone` صراحةً في إعدادات الـ `workflow`، ووثقها.

5.4 المصفوفات — من النسخ إلى الفهم

🧠 هذا القسم مخصص لمن يعتمد على نسخ الكود من **AI**. هذه الدوال الأربع تغطي 90% مما تحتاجه، وفهمها يحركك من الاعتماد.

💡 الدوال الأربع الأساسية

```
const items = [
  { name: "أحمد", score: 80 },
  { name: "سارة", score: 95 },
  { name: "خالد", score: 60 }
];
```

الدالة	السؤال الذي تجيب عنه	المثال	النتيجة
<code>.map()</code>	"حوّل كل عنصر إلى شيء آخر"	<code>items.map(i => i.name)</code>	<code>["أحمد", "سارة", "خالد"]</code>
<code>.filter()</code>	"أبقي ما يحقق الشرط"	<code>items.filter(i => i.score > 70)</code>	أحمد وسارة
<code>.find()</code>	"أعطني أول ما يحقق الشرط"	<code>items.find(i => i.score > 90)</code>	سارة (كائن واحد)
<code>.reduce()</code>	"اجمعها كلها في قيمة واحدة"	<code>items.reduce((s,i) => s + i.score, 0)</code>	235

⚙️ كيف تقرأها؟ — المفتاح

```
items.filter(i => i.score > 70)
|           |           |
|           |           | الشرط
|           |           | بحيث " (السهم)
|           |           | اسم مؤقت للعنصر الواحد (سجّه ما شئت)
|           |           | العملية
```

اقرأها بالعربية: "من `items`، أبقي كل عنصر `i` بحيث `i.score` أكبر من 70."

⚠️ `i` ليست كلمة سحرية. هي اسم مؤقت تختاره أنت. `items.filter(lead => lead.score > 70)` نفس الشيء تمامًا وأوضح. نصيحة: سم المتغير بما يمثله (`user` , `order` , `lead`) لا `i` — يجعل الكود مقروءًا لك بعد شهر.

⚠️ الفرق الذي يوقع الجميع: `find` مقابل `filter`

يُرجع	إن لم يجد
<code>.find()</code>	كائنًا واحدًا
<code>.filter()</code>	مصفوفة (ولو بعنصر واحد)
	<code>undefined</code>
	<code>[]</code> مصفوفة فارغة

```
items.find(i => i.name === "سارة").score // 95 ✓
items.filter(i => i.name === "سارة").score // undefined ✗ (score المصفوفة لا تملك)
items.filter(i => i.name === "سارة")[0].score // 95 ✓
```

🐛 هذا سبب شائع جدًا لـ `Cannot read property of undefined`. اسأل نفسك دائمًا: هل أتعامل مع كائن أم مصفوفة؟

Challenge 5.4 🏆

من items أعلاه، اكتب تعبيرًا يُرجع أسماء من تجاوز 70 فقط، مفصولة بفاصلة: أحمد , سارة

الحل ▼

```
{{ $json.items.filter(p => p.score > 70).map(p => p.name).join(', ') }}
```

التسلسل: filter يصفّي map يحوّل لأسماء → join يدمج بنص. الفكرة المهمة: الدوال تتسلسل. كل واحدة تستقبل نتيجة سابقتها. هذا نمط تستخدمه يوميًا.

5.5 الشروط داخل التعبيرات

الشروط الثلاثي (Ternary)

```
{{ $json.score > 70 ? 'غير مؤهل' : 'مؤهل' }}  
// الشرط ؟ إن صح : إن كذب
```

متداخل (استخدمه بحذر — أكثر من مستويين يصبح غير مقروء):

```
{{ $json.score > 90 ? 'ساخن' : $json.score > 60 ? 'بارد' : 'دافئ' }}
```

★ متى تستخدم Ternary ومتى IF Node؟

- Ternary: لتحديد قيمة (نص، رقم) — البيانات تكمل بنفس المسار.
- IF Node: لتحديد مسار — بيانات مختلفة تذهب لعقد مختلفة.

استخدام Ternary لاتخاذ قرارات مسار معقدة يجعل ال workflow غير مقروء. الوضوح البصري أهم من الاختصار.

المعاملات المفيدة

المعامل	المعنى	مثال
	"أو" — قيمة افتراضية	{{ \$json.name 'زائر' }}
??	مثله لكن يستبدل عند undefined / null فقط	{{ \$json.count ?? 0 }}
?.	وصول آمن	{{ \$json.a?.b }}

⚠️ الفرق الدقيق بين `||` و `??` مهم: `{{ $json.count || 10 }}` → إن كان `count = 0` يُرجع `10` (لأن `0` يُعتبر "كاذبًا")
`{{ $json.count ?? 10 }}` → إن كان `count = 0` يُرجع `0` (لأنه ليس `null`)
 متى يؤديك هذا؟ عند التعامل مع أرقام قد تكون صفرًا بشكل مشروع: كمية، رصيد، عدد مرات. استخدم `??` معها.

5.6 تشخيص التعبيرات

📋 جدول الأخطاء الشائعة

رسالة الخطأ	السبب	الحل
Cannot read property 'x' of undefined	مسار غير موجود	<code>?</code> + افحص Input فعليًا
x is not a function	نوع خاطئ (نص عومل كمصفوفة)	تحقق من النوع
يظهر النص حرفيًا	نسيان <code>{{ }}</code> أو الحقل نمي بحت	فعل وضع Expression
undefined في المخرج	اسم حقل خاطئ (حساس لحالة الأحرف)	انسخ الاسم من لوحة Input
يعمل لبعض الـ items	بيانات غير متسقة	تحقق من كل الحالات

🧠 منهجية التشخيص الصحيحة

لا تخمن. جرّي.

التعبير `{{ $json.data.users[0].profile.email }}` لا يعمل؟ فكّكه تدريجيًا:

1. `{{ $json }}` ← ماذا يوجد أصلًا؟

2. `{{ $json.data }}` ← هل `data` موجود؟

3. `{{ $json.data.users }}` ← هل مصفوفة فعلاً؟

4. `{{ $json.data.users[0] }}` ← هل فيها عنصر؟

5. ... حتى تجد أول مستوى يُرجع `undefined` — هناك المشكلة بالضبط.

هذه المنهجية تُسمى التنبيف (Bisection)، وهي أهم مهارة تشخيص في البرمجة كلها. من يستخدمها لا يحتاج سؤال AI في 90% من الحالات. من لا يستخدمها يبقى عالقًا للأبد.

Common Mistakes — Module 5 ⚠️

الخطأ	التصحيح
<code>\$json</code> للوصول لعقدة بعيدة	<code>\$node["الاسم"].json</code>
تغيير اسم عقدة بعد الإشارة إليها	راجع كل التعبيرات المرتبطة
<code>.filter()[0]</code> منسّي	<code>filter</code> يُرجع مصفوفة
ـ	
تجاهل المنطقة الزمنية	اضبطها صراحةً
Ternary متداخل معقد	استخدم Switch Node
التخمين بدل التجزئة	فكّك التعبير مستوى مستوى

Mini Quiz — Module 5 📝

1. الفرق بين `$json` و `$node["X"].json` ؟
2. `{{ $json.qty ?? 5 }}` و `qty = 0` — الناتج؟ ولماذا؟
3. اكتب تعبيرًا يُرجع تاريخ بعد 7 أيام بصيغة `yyyy-MM-dd`.
4. `{{ $json.list.find(x => x.id === 3).name }}` انهار. لماذا؟ كيف تحميه؟
5. متى Ternary ومتى IF Node؟

الإجابات ✓ ▼

1. `$json` = بيانات الـ item الحالي من العقدة السابقة مباشرة. `$node["X"].json` = بيانات من عقدة محددة بالاسم مهما بعدت.
2. `0` — لأن `??` يستبدل فقط عند `null` أو `undefined`، و `0` ليس أيًا منهما. (لو كان `||` لأرجع 5).
3. `{{ $now.plus({ days: 7 }).toFormat('yyyy-MM-dd') }}`
4. لأن `find` أرجع `undefined` (لا يوجد عنصر بـ `id=3`)، والوصول إلى `.name` من `undefined` ينهار. الحماية: `{{ $json.list.find(x => x.id === 3)?.name || 'غير موجود' }}`
5. Ternary لاختيار قيمة ضمن نفس المسار. IF Node لاختيار مسار تسلكه البيانات.

Definition of Done — Module 5 ✓

- ☐ أكتب تعبيرًا للوصول لأي بيانات دون نسخ.
- ☐ أستخدم `$node[]` للوصول لعقد بعيدة.
- ☐ أتعامل مع التواريخ والمناطق الزمنية بوعي.
- ☐ أشرح `map` و `filter` و `find` و `reduce` بكلماتي — لا أنسخها.
- ☐ أفرق بين `||` و `??` وأعرف متى يؤدي الخلط.

- ☐ أشخص تعبيراً فاشلاً بالتجزئة دون سؤال AI.

Module 6 — Core Nodes

الهدف: إتقان العقد التي تشكل 90% من أي workflow. المدة: 150 دقيقة · المتطلبات: Modules 4, 5

📌 كل الإصدارات أدناه مُتحققة من نسخة n8n حقيقية (2.37.10).

Edit Fields (Set) 6.1

📌 n8n-nodes-base.set — v3.5 · الأوصاع: manual (تعيين حقل بحقل) · raw (JSON كامل)

البند	التفصيل
ما هي	تُنشئ أو تُعدل أو تُحذف حقولاً
لماذا توجد	لتشكيل البيانات بالصورة التي تتوقعها العقدة التالية
متى	تنظيف مخرجات API · توحيد الأسماء · إضافة بيانات وصفية

★ الممارسة الأهم: ضع عقدة Set بعد كل مصدر بيانات خارجي لتوحيد الشكل. لماذا؟ إن تغيّر الـ API لاحقاً، تعدّل عقدة واحدة بدل عشرين تعبيراً موزعة في الـ workflow. هذا يُسمى طبقة مقاومة التغيير (Anti-Corruption Layer)، وهو نمط معماري حقيقي — وهو الفرق بين نظام يُصان ونظام يُعاد بناؤه.

⚠️ خيار "Include Other Input Fields": إن كان مطفاً، تُحذف كل الحقول غير المذكورة صراحةً. هذا سبب متكرر لسؤال "أين اختفت بياناتي؟"

IF 6.2

📌 n8n-nodes-base.if — v2.3

البند	التفصيل
ما هي	توجّه الـ items لمسارين: true و false
متى	قرار ثنائي واضح

⚠️ تحذير خرج مُتحقق من مصدر n8n نفسه: IF له مخرجان. إن تركت أحدهما غير موصول، تُسقط الـ items المتجهة إليه بصمت — بلا خطأ وبلا تنبيه.

هذا أخطر سلوك في n8n للمبتدئين، لأنه لا يظهر كخطأ إطلاقاً. البيانات ببساطة تختفي. القاعدة: بعد كل IF أسأل: "أين تذهب الـ items في الفرع الآخر؟" إن كان الجواب "لا يهمني" — وثق ذلك بـ Sticky Note حتى لا يظن من يصون النظام لاحقاً أنه عطل.

⚠️ فخ نوع البيانات

الشرط `budget > 50000` يفشل إن كانت `budget` نصاً `"50000"`. الحل: `{{ Number($json.budget) }}` أو استخدم خيار التحقق الصارم من النوع في العقدة.

Switch 6.3

n8n-nodes-base.switch — v3.4 · الأوضاع · rules · expression 📌

البند	التفصيل
ما هي	توجيه لعدة مسارات (أكثر من اثنين)
متى	تصنيف متعدد: نوع الطلب، حالة العميل، أولوية

★ **فعل دائماً مخرج Fallback** للحالات غير المتوقعة. بدونه، أي قيمة جديدة لم تخطر ببالك (وستأتي — دائماً تأتي) تختفي بصمت. مثال واقعي: بنييت Switch لحالات جديد / مؤهل / مرفوض. أضاف العميل حالة مؤجل في نظامه. بلا Fallback، كل العملاء المؤجلين يختفون من نظامك دون أن يلاحظ أحد لأسابيع.

Filter 6.4

n8n-nodes-base.filter — v2.3 📌

البند	التفصيل
ما هي	تُبقي الـ items المطابقة وتحذف الباقي
الفرق عن IF	Filter مخرج واحد (حذف)، IF مخرجان (توجيه)

🔍 مُتحقق: Filter يُخرج 0 items عند عدم التطابق، والسلسلة تتوقف بنظافة — لا تحتاج بوابة IF قبل الحلقات للتحقق من وجود عناصر.

⚠️ اختيار الوضع هو أخطر قرار في هذه العقدة. الوضع الخاطئ لا يُنتج خطأً — بل يُسقط أو يُكرّر الـ items بصمت. مُتحقّق من مصدر n8n.

الوضع	ماذا يفعل	متى تستخدمه
append	يضع items المدخلات واحدًا تلو الآخر	دمج نتائج فروع متوازية في قائمة
combine → by position	يدمج حسب الترتيب	فقط إن تطابق العدد والترتيب في الفرعين
combine → by fields	يدمج بمفتاح مشترك	الأشيع والأصح غالبًا: "ادمج بالـ id"
combineBySql	استعلام SQL على المدخلات	أكثر من مدخلين أو تجميعات
chooseBranch	يُخرج فرعًا واحدًا ويهمل الباقي	اختيار مسار بعد شرط

🐛 أشهر خطأ: استخدام by position لدمج بيانات يُفترض ربطها بمعرّف. إن أعاد الفرع الأول 10 items والثاني 8، تُدمج بترتيب خاطئ وتُنتج بيانات مغلوطة تبدو سليمة. القاعدة: إن كان لديك معرّف مشترك، استخدم by fields دائمًا.

💡 Merge تجمع من فروع مختلفة. Aggregate تجمع من فرع واحد. لا تخلط بينهما.

Loop Over Items (Split in Batches) 6.6

💡 متى تحتاجها فعلاً؟

⚠️ معظم المبتدئين يستخدمونها بلا داع. n8n يكرّر تلقائيًا على كل item. لا تحتاج حلقة للعمليات العادية.

تحتاجها فقط عند:

الحالة	السبب
Rate Limiting	API يسمح بـ 10 طلبات/دقيقة → دفعات مع Wait
معالجة تراكمية	كل دورة تبني على سابقتها
حجم كبير	item 100,000 تستنزف الذاكرة
منطق تسلسلي	الترتيب مهم والنتيجة تعتمد على السابق

النمط الصحيح ⚙️

```
[Loop Over Items] —"loop"→ [العمل] → (Loop Over Items عُد إلى)
|
└─"done"→ [ما بعد اكتمال الكل]
```

🔍 **مُتَحَقِّق:** مخرج done يعمل تلقائيًا بعد انتهاء كل ال items. والعقدة لا تفعل شيئًا عند دخول items 0 — لا تضيف IF قبلها للتحقق من وجود بيانات، فهذا تعقيد بلا فائدة.

🐛 **أشهر خطأ:** نسيان توصيل مخرج العمل رجوعًا إلى عقدة الحلقة. النتيجة: تُعالج الدفعة الأولى فقط ويتوقف كل شيء.

Code Node 6.7

runOnceForEachItem · runOnceForAllItems · الأوضاع: v2 — n8n-nodes-base.code 📌

⚠️ ⚠️ أهم تحذيرين في هذا المنهج كله — كلاهما مُتَحَقِّق من مصدر n8n:

1. ال Code Node لا يملك وصولًا للشبكة، إطلاقًا. (`fetch()` و `axios` و `XMLHttpRequest` واستيراد وحدات HTTP — كلها تفشل وقت التشغيل. لا تطلب من AI كودًا يستدعي API داخل Code Node — سيكتبه لك وسيفشل. استخدم **HTTP Request Node** ثم عالج مخرجاته في Code.
2. ال Code Node هو الملاذ الأخير، لا الأول. يعمل في بيئة معزولة وأبطأ من العقد الأصلية.

⚙ متى تستخدم عقدة أصلية بدل الكود؟

تريد	استخدم	لا تستخدم
تعديل حقول	Set	Code
تصفية	Filter	Code
توجيه	IF / Switch	Code
تفكيك مصفوفة	Split Out	Code
تجميع	Aggregate	Code
حذف مكرر	Remove Duplicates	Code
تواريخ	Date & Time	Code
أي استدعاء HTTP	HTTP Request	Code ← مستحيل تقنيًا

استخدم Code فقط عند: خوارزمية متعددة الخطوات · تحويل بنية معقدة · منطق لا يُعبّر عنه بتعبير واحد.

⚙ الوضعان

```
// Run Once for All Items – ترى كل شيء مرة واحدة،
const items = $input.all();
const total = items.reduce((sum, item) => sum + item.json.amount, 0);
return [{ json: { total, count: items.length } }];
```

```
// Run Once for Each Item – item مرة لكل
const data = $input.item.json;
return { json: { ...data, fullName: `${data.first} ${data.last}` } };
```

⚠ الشكل المُعاد إلزامي: مصفوفة من كائنات، كل كائن فيه مفتاح json. return [{ json: { ... } }]. return { ... } أشهر خطأ في هذه العقدة.

🐛 Debugging داخل Code

```
console.log('قيمة المتغير:', someVar); // تظهر في لوحة المخرجات
```

🔍 هذه أبسط أداة تشخيص وأكثرها إهمالاً. عند الشك في قيمة، اطبعها — لا تخمّن.

6.8 عقد أساسية أخرى

العقدة	typeVersion	الاستخدام
HTTP Request	v4.5	أي 7 Module — API
Wait	v1.1	تأخير · Rate Limiting
Remove Duplicates	v2	حذف مكرر · ومنع التكرار عبر التنفيذات
Date & Time	v2	عمليات التاريخ
Split Out	v1	مصفوفة → items
Aggregate	v1	items → مصفوفة
Limit	v1	تحديد العدد (مفيد للاختبار)
Compare Datasets	v2.3	مقارنة مجموعتين لكشف التغييرات
Execute Sub-workflow	v1.3	استدعاء workflow آخر — Module 15
Respond to Webhook	v1.5	الرد على webhook

 **جوهره مخفية — Remove Duplicates:** له عملية `removeItemsSeenInPreviousExecutions` — تحذف ما رآه النظام في تنفيذه سابقة، لا في التنفيذ الحالي فقط. هذا حل جاهز لمشكلة **Idempotency** (منع المعالجة المكررة) التي تُبنى عادةً بجدول قاعدة بيانات معقدة. استخدامه المثالي: webhook قد يصل مرتين، أو polling قد يعيد نفس السجل. راجع Module 13.

Common Mistakes — Module 6 ⚠️

الخطأ	الأثر	التصحيح
مخرج IF غير موصول	اختفاء صامت للبيانات	وَصِّل الفرعين أو وُثِّق السبب
وضع Merge خاطئ	تكرار أو فقدان صامت	by fields عند وجود مفتاح
Switch بلا Fallback	القيم الجديدة تختفي	Fallback فعَل
حلقة بلا داع	تعقيد وبطء	n8n يكرر تلقائيًا
نسيان الرجوع في الحلقة	دفعة واحدة فقط	وَصِّل العمل رجوعًا للحلقة
fetch() في Code	فشل مؤكد	HTTP Request Node
شكل return خاطئ	خطأ فورى	<code>[{ json: {...} }]</code>
Set بلا Include Other Fields	اختفاء الحقول	فعَل الخيار

Mini Quiz — Module 6 📝

1. IF بفرع غير موصول — ماذا يحدث لك items؟
2. متى by fields ومتى by position في Merge؟
3. لماذا لا يعمل fetch() في Code Node؟
4. لديك 500 item وAPI يسمح بـ 10/دقيقة. ما البنية؟
5. ما شكل return الصحيح في Code (وضع All Items)؟
6. أي عقدة تمنع معالجة نفس السجل مرتين عبر تنفيذات مختلفة؟

الإجابات ▼

1. تُسْقَط بصمت — بلا خطأ وبلا تنبيه. من أخطر السلوكيات لأنه غير مرئي.
2. by fields عند وجود مفتاح مشترك (مثلًا id) — الأصح غالبًا. by position فقط عند ضمان تطابق العدد والترتيب في الفرعين.
3. لأن بيئة الـ Code Node معزولة بلا وصول للشبكة. الحل: HTTP Request Node ثم معالجة مخرجاته.
4. Loop Over Items (حجم الدفعة 10) → HTTP Request → Wait (60 ثانية) → رجوع للحلقة.
5. `return [{ json: { ... } }]` — مصفوفة من كائنات كل منها يحوي مفتاح json.
6. Remove Duplicates بعملية `removeItemsSeenInPreviousExecutions`.

Definition of Done — Module 6 ✓

- ☐ استخدم Set لتوحيد شكل البيانات بعد كل مصدر خارجي.
- ☐ أوَصِّل مخرجي IF دائمًا أو أوُثِّق سبب عدم التوصيل.

- ☐ أختار وضع Merge بوعي وأشرح سبب الاختيار.
 - ☐ لا أستخدم حلقة إلا بمبرر من الحالات الأربع.
 - ☐ أعرف يقيناً أن **Code Node** لا يصل للشبكة.
 - ☐ أفضل العقد الأصلية على الكود.
 - ☐ بنيث workflow يستخدم Set + IF + Merge + Split Out معاً.
-

المستوى 3 — Integration

Module 7 — APIs

الهدف: أن تربط أي خدمة في العالم، حتى لو لم تكن لها عقدة في n8n. المدة: 150 دقيقة · المتطلبات: Modules 4, 5

المهارات المكتسبة

- قراءة توثيق أي API واستخراج ما تحتاجه منه
- بناء الطلبات بأنواع المصادقة المختلفة
- قراءة رموز الحالة والتصرف بناءً عليها
- التعامل مع Rate Limits و Pagination

7.1 ما هي الـ API؟

💡 الفكرة ببساطة

الـ API هي قائمة طعام المطعم: توضح ما يمكنك طلبه وكيف تطلبه. لست بحاجة لدخول المطبخ. الخدمة تقول: "إن أرسلت لي طلبًا بهذا الشكل على هذا العنوان، سأعيد لك نتيجة بهذا الشكل."

🧠 تحوّل التفكير الأهم في هذا المنهج

المبتدئ: "هل يوجد Node لخدمة X؟" المحترف: "هل لخدمة X واجهة API؟" — إن نعم، فهي مدعومة. عقد الخدمات الجاهزة اختصار للراحة، لا شرط للربط. هذا التحول وحده يوسع قدرتك من 400 خدمة إلى كل خدمة على الإنترنت، ويجعلك قادرًا على قبول مشاريع يرفضها منافسوك.

7.2 تشريح طلب HTTP

كل طلب في العالم يتكون من خمسة أجزاء:

1. METHOD	POST	ما نوع العملية؟
2. URL	https://api.x.com/v1/leads	أين؟
3. HEADERS	Authorization: Bearer xxx Content-Type: application/json	من أنا؟ وبأي صيغة؟
4. QUERY	?limit=50&status=active	فلاتر وخيارات
5. BODY	{"name": "محمد", "budget": 75000}	البيانات المُرسلة

⚙️ الأفعال (HTTP Methods)

العمل	المعنى	آمن للتكرار؟	مثال
GET	اقرأ	✓ نعم	جلب قائمة العملاء
POST	أنشئ	✗ لا	إنشاء عميل جديد
PUT	استبدل كاملاً	✓ نعم	استبدال سجل كامل
PATCH	عدّل جزئياً	✓ عادةً	تحديث حقل واحد
DELETE	احذف	✓ نعم	حذف سجل

⚠️ عمود "آمن للتكرار" (Idempotency) هو الأهم عملياً. إعادة محاولة POST قد تُنشئ سجلاً مكرراً. إعادة PUT تُنتج نفس النتيجة. الأثر التصميمي: عند إعداد Retry تلقائي، انتبه أن POST قد يُنشئ نسخاً متعددة. هذا ليس تفصيلاً نظرياً — هو سبب حقيقي لظهور طلبات مكررة في أنظمة الإنتاج.

⚙️ Query Parameters مقابل Body

Body	Query	
في جسم الطلب	في الرابط بعد ؟	أين
POST / PUT / PATCH	غالبًا GET	مع أي فعل
البيانات المُرسلة	تصفية، ترقيم، فرز	الاستخدام
✗ لا	✓ نعم	مرئي في السجلات

🔒 لا تضع بيانات حساسة في Query Parameters أبداً. الروابط تُسجل في سجلات الخوادم والوسطاء وتاريخ المتصفح. الأسرار في Headers فقط.

7.3 رموز الحالة (Status Codes)

💡 القراءة السريعة بالخانة الأولى

المدى	المعنى	من المسؤول؟
2xx	نجاح ✓	—
3xx	إعادة توجيه	—
4xx	خطوك أنت	راجع طلبك
5xx	خطوهم هم	أعد المحاولة لاحقاً

⚙️ الرموز التي ستقابلها فعلاً

الرمز	المعنى	التشخيص
200	نجاح	—
201	أنشئ	—
204	نجاح بلا محتوى	طبيعي في DELETE
400	طلب سيئ	البيانات المرسله خاطئة الشكل
401	غير مصادق	المفتاح خاطئ أو منته
403	ممنوع	مصادق لكن بلا صلاحية
404	غير موجود	تحقق من الرابط والمعرف
409	تعارض	السجل موجود مسبقاً
422	كيان غير قابل للمعالجة	الشكل صحيح لكن القيم غير صالحة
429	تجاوز الحد	أبطل — راجع §7.6
500	خطأ خادم	أعد المحاولة
502/503/504	الخدمة غير متاحة	أعد المحاولة بتباعد متزايد

📌 الفرق بين 401 و 403 يوفر ساعات: 401 = "من أنت؟" ← المفتاح خاطئ أو منته أو غير مرسل. 403 = "أعرفك، لكن لا يُسمح لك بهذا" ← المفتاح صحيح لكن الصلاحيات ناقصة. من يخلط بينهما يقضي ساعة يجدد مفتاحاً سليماً، بينما المشكلة في نطاق الصلاحيات (Scopes).

7.4 المصادقة (Authentication)

⚙️ الأنواع الأربعة

النوع	الشكل	الأمان	الشيوع
API Key	X-API-Key: abc123 أو في Query	متوسط	شائع جدًا
Bearer Token	Authorization: Bearer eyJ...	جيد	الأشيع
Basic Auth	Authorization: Basic <base64>	ضعيف (يتطلب HTTPS)	قديم
OAuth 2.0	تدفق تفويض متعدد الخطوات	الأقوى	خدمات كبرى

💡 OAuth 2.0 مبسطًا

المشكلة التي يحلها: كيف تمنح تطبيقًا وصولًا لبياناتك دون إعطائه كلمة مرورك؟

التشبيه: مفتاح صف السيارات (Valet Key) — يفتح الباب ويشغل المحرك، لكنه لا يفتح الصندوق ولا يعمل لأكثر من ساعة.

1. يوجّهك لصفحة تسجيل دخول الخدمة n8n
2. تسجّل دخولك وتوافق على الصلاحيات المطلوبة
3. رمزًا مؤقتًا n8n الخدمة تعطي (code)
4. Access Token (+ Refresh Token) يبادله بـ n8n
5. في الطلبات Access Token يستخدم الـ n8n
6. Refresh Token عند انتهاء صلاحيته، يجدّده تلقائيًا بالـ

🔑 عمليًا في n8n: أنشئ Credential من النوع OAuth2، اضغط "Connect"، أكمل التفويض. n8n يدير الخطوات 3-6 تلقائيًا.

⚠️ متى ينكسر OAuth؟ عند تغيير كلمة المرور، أو إلغاء الوصول، أو انتهاء صلاحية Refresh Token (بعض الخدمات تُنهيه بعد فترة خمول).
العرّض: 401 مفاجئ في نظام كان يعمل شهرًا. للعملاء: وثّق أي حساب رُبط، ونبّه العميل ألا يلغي الوصول. أضف تنبيهًا عند 401 (Module 11).

🔒 قاعدة غير قابلة للتفاوض

استخدم دائمًا ميزة Credentials المدمجة في HTTP Request Node — لا تكتب Authorization يدويًا في Header. السبب: القيمة المكتوبة يدويًا تُخزّن نصًا صريحًا داخل تعريف الـ workflow، فتُصدّر وتُشارك وتُنسخ احتياطيًا معه.

7.5 HTTP Request Node عمليًا

📌 n8n-nodes-base.httpRequest — v4.5

Method: GET
URL: https://api.github.com/users/n8n-io

نفذ، ثم افحص المخرجات. لاحظ بنية JSON المُعادة.

ثم أضف Query Parameters:

URL: https://api.github.com/users/n8n-io/repos
Query: per_page = 5
sort = updated

ثم حوّل النتيجة إلى items منفصلة (تذكّر Module 4): هل احتجت Split Out؟ افحص عدد الـ items أولاً.

 الإعدادات المتقدمة المهمة

متى تفعله	الإعداد
لقراءة حدود المعدل من الـ headers	Response → Include Response Headers
لمعالجة رموز 4xx بنفسك بدل إيقاف الـ workflow	Response → Never Error
عندما تُرجع الخدمة نتائج مقسّمة على صفحات	Pagination
للتحكم في معدل الإرسال	Batching
لمنع التعليق اللانهائي	Timeout
لإعادة المحاولة تلقائياً	Retry On Fail (في إعدادات العقدة)

★ مُتَحَقِّق من مصدر n8n: فضّل العقد المخصصة على HTTP Request عند توفرها — فيها مصادقة مدمجة ومعالجة أخطاء أفضل وصيانة أسهل. استخدم HTTP Request عند: عدم وجود عقدة API داخلي · عملية لا تدعمها العقدة المخصصة.

Rate Limits 7.6 — الدرس الذي يتعلمه الجميع بالطريقة الصعبة

 الفكرة

كل API يحدّ عدد الطلبات المسموحة في فترة. عند التجاوز: **429 Too Many Requests**.

 لماذا يهم؟

تجاوز الحد بشكل متكرر قد يؤدي إلى حظر مؤقت أو دائم للحساب. إن كان حساب عميل — فهذه مشكلة مهنية، لا تقنية فقط.

الاستراتيجية	التنفيذ	متى
التباطؤ الاستباقي	Loop + Wait بحجم دفعة أقل من الحد	تعرف الحد مسبقاً
التراجع الأسّي	Retry بتباعد متزايد: 1s → 2s → 4s → 8s	فشل عابر
احترام Retry-After	اقرأ ال header وانتظر المدة المطلوبة	الخدمة تخبرك بالمدة
التخزين المؤقت	خزن النتائج المتكررة	بيانات لا تتغير كثيراً

النمط العملي للتباطؤ 🔧

```
(حجم الدفعة = الحد - هامش أمان) [Loop Over Items]
|
├─ loop → [HTTP Request] → [Wait 60s] → (رجوع للحلقة)
|
└─ done → [المتابعة]
```

🐛 بروتوكول تشخيص 429:

1. افحص Executions واقرأ الرد كاملاً (لا رمز الحالة فقط) — كثير من الخدمات تُرجع الحد المتبقي ووقت إعادة التعيين.
 2. افحص response headers: X-RateLimit-Remaining و Retry-After .
 3. احسب معدلك الفعلي: كم طلباً يرسل نظامك في الدقيقة؟
 4. قارنه بالحد الموثق.
 5. صمم التباطؤ بناءً على الرقم، لا بالتخمين.
- لاحظ: إضافة Retry بشكل أعمى تزيد المشكلة سوءاً — أنت ترسل طلبات أكثر لخدمة تقول لك "أنت ترسل كثيراً". هذا خطأ شائع جداً.

Pagination 7.7

💡 الفكرة

ال API لا يعيد 10,000 سجل دفعة واحدة. يعيدها صفحات.

النمط	الآلية	كيف تعرف أنك انتهيت
Offset/Limit	?offset=0&limit=100	صفحة أقصر من الحد أو فارغة
Page Number	?page=1&per_page=100	صفحة فارغة
Cursor	?cursor=abc123	لا يوجد cursor في الرد
Link Header	رابط الصفحة التالية في ال header	لا يوجد "rel=next"

🔗 HTTP Request Node يدعم Pagination مدمجًا في قسم **Pagination** — استخدمه بدل بناء حلقة يدوية. ⚠️ ضع دائمًا حدًا أقصى لعدد الصفحات. خطأ في شرط التوقف = حلقة لا نهائية تستهلك حمتك وقد تحظر حسابك.

7.8 كيف تقرأ توثيق API لم تره من قبل؟

🧠 هذه المهارة أهم من حفظ أي API بعينه. هي التي تجعلك مستقلًا.

البروتوكول — سبع خطوات بالترتيب:

#	ابحث عن	السؤال
1	Base URL	ما جذر العنوان؟
2	Authentication	أي نوع؟ وأين يوضع المفتاح؟
3	Endpoints	ما المسار للعملية التي أريدها؟
4	Request Example	ما شكل البيانات المطلوب بالضبط؟
5	Response Example	ما شكل الرد؟ (يحدد تعبيراتك لاحقًا)
6	Rate Limits	ما الحد؟
7	Errors	ما رموز الأخطاء الخاصة بهم؟

★ اختبر خارج n8n أولاً. استخدم curl أو Postman للتأكد من نجاح الطلب قبل بنائه في n8n. لماذا؟ يفصل مشكلة "ال API نفسه" عن مشكلة "إعدادي في n8n" — وهذا عزل المتغيرات، أهم مبدأ في التشخيص. من بيني مباشرة في n8n يواجه مشكلتين محتملتين في وقت واحد ولا يعرف أيهما السبب.

Challenge 7.8 🏆

اختر API عامًا لم تستخدمه من قبل. اقرأ توثيقه بالبروتوكول أعلاه، ثم:

1. اجعله يعمل بـ curl.

2. انقله إلى HTTP Request Node.

3. حوّل النتيجة إلى items منفصلة.

4. صَفِّ النتائج بشرط.

لا تستخدم AI في هذا التحدي. الهدف بناء عضلة قراءة التوثيق، وهي عضلة لا تنمو بالاستعانة.

Common Mistakes — Module 7 ⚠️

الخطأ	التصحيح
مفتاح مكتوب يدويًا في Header	استخدم Credentials
أسرار في Query Parameters	ضعها في Headers
تجاهل Rate Limits	صمّم التباطؤ من البداية
Retry أعمى على 429	اقرأ Retry-After واحترمه
الخلط بين 401 و 403	401 هوية · 403 صلاحية
نسيان Pagination	ستحصل على أول صفحة فقط وتظنّها كل البيانات
Retry على POST بلا حذر	قد يُنشئ سجلات مكررة
البناء في n8n قبل اختبار الـ API	اختبر بـ curl أولاً

Mini Quiz — Module 7 🖋️

1. الفرق بين 401 و 403 وكيف يغيّر تشخيصك؟

2. لماذا لا نضع الأسرار في Query Parameters؟

3. API يسمح بـ 100 طلب/دقيقة ولديك 1000 سجل. صمّم البنية.

4. أي الأفعال آمنة للتكرار ولماذا يهم عند إعداد Retry؟

5. جلبت بيانات فوجدت 100 سجل فقط والخدمة تحوي 5000. ما السبب؟

6. لماذا نختبر بـ curl قبل n8n؟

1. 401 = فشل التعرف على الهوية (مفتاح خاطئ/منتهٍ/مفقود) → أصلح المفتاح. 403 = الهوية معروفة لكن الصلاحية ناقصة → أصلح ال-Scopes. الخلط يجعلك تجد مفتاحًا سليمًا بلا فائدة.
2. لأن الروابط تُسجّل في سجلات الخوادم والوسطاء وتاريخ المتصفح، فيُسَرَّب السر.
3. Loop Over Items بحجم دفعة 90 (هامش أمان) → HTTP Request → Wait 60s → رجوع للحلقة. أو Batching المدمج في HTTP Request.
4. GET و PUT و DELETE وعادةً PATCH آمنة. POST غير آمن — إعادة محاولته قد تُنشئ سجلًا مكرّرًا، فانتبه عند تفعيل Retry التلقائي.
5. Pagination — حصلت على الصفحة الأولى فقط. فعّل Pagination في العقدة.
6. لعزل المتغيرات: يفصل مشكلة ال-API عن مشكلة إعدادك في n8n، فتواجه مشكلة واحدة لا اثنتين.

Definition of Done — Module 7 ✓

- ☐ أقرأ توثيق API جديد وأستخرج ما أحجّاه بالبروتوكول.
- ☐ بنيت استدعاءً ناجحًا بمصادقة عبر Credentials.
- ☐ أقرأ رموز الحالة وأشخص بناءً عليها.
- ☐ أتعامل مع Rate Limits بتصميم استباقي.
- ☐ فعّلت Pagination و جلبت كل البيانات.
- ☐ ربطت API لم أستخدمه من قبل دون مساعدة AI.

Module 8 — Integrations

الهدف: أن تصمّم التكامل نفسه، لا أن تحفظ العقد. المدة: 120 دقيقة · المتطلبات: Module 7

8.1 المبدأ الحاكم

🧠 لا تتعلم العقد. تعلّم أنماط التكامل.

400 عقدة يستحيل حفظها، وتتغير باستمرار. لكن أنماط التكامل لا تتجاوز خمسة، وهي ثابتة عبر كل الخدمات. أتقنها فتتعامل مع أي خدمة، جديدة كانت أو قديمة.

النمط	التدفق	مثال
1. Fetch & Store	مصدر → تحويل → مخزن	API → قاعدة بيانات
2. Event & React	حدث → قرار → إجراء	webhook → تصنيف → إشعار
3. Sync	نظام أ ↔ نظام ب	CRM ↔ جدول بيانات
4. Enrich	بيانات + مصدر خارجي → بيانات أغنى	بريد → بحث → ملف كامل
5. Notify	شرط → رسالة	مخزون منخفض → تنبيه

8.2 بروتوكول تصميم أي تكامل

تسع خطوات — نَقْذها بالترتيب قبل لمس أي عقدة:

#	الخطوة	السؤال
1	الغرض	ما القيمة التجارية بجملة واحدة؟
2	الاتجاه	أحادي أم ثنائي؟ (الثنائي أصعب بكثير — تجنبه إن أمكن)
3	المشغل	Push أم Poll؟
4	شكل البيانات	اطلب عينة حقيقية، لا وصفاً
5	الربط	ما الحقل الذي يربط السجلات بين النظامين؟
6	التعارض	إن تغيّر السجل في الطرفين، من يفوز؟
7	الحجم	كم سجلاً يومياً؟ ذروة؟
8	الفشل	ماذا يحدث إن توقف أحد الطرفين؟
9	التكرار	كيف أ منع المعالجة المزدوجة؟

⚠️ الخطوتان 5 و 6 هما ما ينسى الجميع، وهما سبب معظم كوارث المزامنة. بلا حقل ربط موثوق، سننشئ سجلات مكررة. بلا قاعدة لحسم التعارض، ستفقد بيانات مامناً.

8.3 اعتبارات لأشهر التكاملات

الإصدارات مُتحقَّقة من نسخة حقيقية.

الخدمة	typeVersion	الاعتبار الحاسم
Google Sheets	v4.7	appendOrUpdate (upsert) يمنع التكرار. الجداول تبطؤ فوق ~10 آلاف صف — انقل لقاعدة بيانات
Gmail	v2.2	حدود إرسال يومية. استخدم Gmail Trigger بدل Schedule+getAll
Telegram	v1.2 · Trigger v1.5	الأسهل للبوتات — بلا موافقات. يدعم sendAndWait
WhatsApp Business	v1.1 · Trigger v1	الأصعب: يتطلب موافقة Meta وقوالب معتمدة. التحقق تلقائي بلا verify token
Postgres	v2.7	upsert مدمج. استخدم الاستعلامات المعاملية
Data Table (مدمج)	v1.1	تخزين دائم بلا قاعدة خارجية — ممتاز للبداية
Discord	v2	يدعم sendAndWait للموافقات

⚠ ملاحظة مُتحقَّقة عن Data Table: لا توجد عملية getAll. لجلب صفوف كثيرة استخدم operation: 'get' مع returnAll: true. ومعرّفات الصفوف تُولّد تلقائيًا — لا تُنشئ عمود id خاصًا بك.

🚀 نصيحة تجارية — واتساب: العملاء يطلبون واتساب دائمًا لأنه الأشهر. لكنه الأصعب إعدادًا (موافقة Meta، قوالب معتمدة، نافذة 24 ساعة للرد الحر). الاستراتيجية: ابن المنطق أولاً على Telegram (إعدادة دقائق)، وأثبت أن النظام يعمل، ثم بَدَل قناة الإرسال إلى واتساب. الميزة المعمارية: إن عزلت الإرسال في sub-workflow منفصل، يصبح التبديل تغيير عقدة واحدة لا إعادة بناء. هذا مثال عملي على فصل الاهتمامات (Module 15).

8.4 التعامل مع خدمة بلا عقدة

البروتوكول:

1. عام؟ — لا —► ابحث عن بديل أو أتمتة جزئية API هل للخدمة
|
نعم
|
2. اقرأ التوثيق (بروتوكول §7.8).
3. curl اختبر بـ
4. n8n في Generic من نوع Credential أنشئ
5. HTTP Request Node ابن
6. قابل لإعادة الاستخدام sub-workflow اعزل الاستدعاء في *
7. أصف معالجة أخطاء ثم وثّق

★ الخطوة 6 هي ما يفعله المحترفون. بدل تكرار استدعاء نفس الـ API في عشرة workflows، اجعله وحدة واحدة تستدعيها. تغيّر الـ API؟ تعدّل مكانًا واحدًا لا عشرة. هذا الفرق بين نظام يُصان ونظام يتعقّن.

Definition of Done — Module 8 ✓

- ☐ • أصف أي تكامل ضمن الأنماط الخمسة.
- ☐ • أطبق بروتوكول التصميم التساعي قبل البناء.
- ☐ • أحدد حقل الربط وقاعدة حسم التعارض قبل أي كود.
- ☐ • ربطتُ خدمة بلا عقدة عبر HTTP Request.
- ☐ • عزلتُ تكاملاً في sub-workflow قابل لإعادة الاستخدام.

Module 9 — قواعد البيانات

الهدف: تصميم مخزن بيانات صحيح وتنفيذ CRUD كامل من n8n. المدة: 180 دقيقة · المتطلبات: Modules 4, 7

9.1 لماذا نحتاج قاعدة بيانات؟

؟ المشكلة

الـ workflow بلا ذاكرة. كل تنفيذ يبدأ من الصفر ولا يعرف شيئاً عن سابقه.

بلا قاعدة بيانات لا تستطيع الإجابة على:

- هل عالجتنا هذا العميل من قبل؟ (منع التكرار)
- ما حالة هذا الطلب الآن؟ (تتبع الحالة)
- كم عميلاً وصلنا هذا الشهر؟ (تحليل)

- ماذا قال هذا العميل في محادثته السابقة؟ (سياق)

اختيار المخزن ⚙️

الخيار	متى	القيود
Data Table (مدمج، v1.1)	نماذج أولية · بيانات بسيطة · بلا بنية خارجية	إمكانات استعلام محدودة
Google Sheets (v4.7)	العميل يريد رؤية البيانات وتحريرها	بطيء، فوق 10 آلاف صف · بلا علاقات
Postgres / Supabase (v2.7)	الخيار الإنتاجي	يحتاج إعدادًا
Vector DB (PGVector v1.3)	البحث الدلالي و RAG	لحالة استخدام محددة

🧠 قاعدة القرار: نموذج أولي أو حجم صغير → **Data Table**. العميل يريد التحرير المباشر → **Sheets**. نظام حقيقي بعلاقات ونمو → **Postgres**.
(Supabase تعطيك Postgres مُدجلاً بخطة مجانية سخية.)

9.2 أساسيات SQL

المفردات 💡

Excel مثل ورقة في — (جدول) Table
| — Column (عمود) — حقل واحد له نوع محدد
| — Row (صف) — سجل واحد
| — Primary Key — معرف فريد لكل صف

أنواع البيانات الأساسية ⚙️

النوع	الاستخدام
<code>VARCHAR(n)</code> / <code>TEXT</code>	نصوص
<code>BIGINT</code> / <code>INTEGER</code>	أعداد صحيحة
<code>DECIMAL(10,2)</code>	مبالغ مالية — لا تستخدم <code>FLOAT</code> للنقود أبدًا
<code>BOOLEAN</code>	صح/خطأ
<code>TIMESTAMP TZ</code>	تاريخ ووقت مع المنطقة الزمنية
<code>JSONB</code>	بيانات JSON مرنة وقابلة للفهرسة
<code>UUID</code>	معرف فريد عالميًا

⚠️ **FLOAT** للنقود يسبب أخطاء تقريب حقيقية ($0.1 + 0.2 \neq 0.3$ في الحساب العشري الثنائي). استخدم **DECIMAL** دائماً للمبالغ. هذا خطأ يظهر بعد أشهر في التقارير المالية ويصعب تتبعه.

⚠️ استخدم **TIMESTAMPZ** لا **TIMESTAMP**. بلا منطقة زمنية، ستواجه اختلافات غامضة عند تغيير الخوادم أو تعدد المستخدمين.

⚙️ العمليات الأربع (CRUD)

```
-- CREATE
INSERT INTO leads (name, email, budget)
VALUES ('محمد', 'm@example.com', 75000);

-- READ
SELECT * FROM leads WHERE budget > 50000 ORDER BY created_at DESC LIMIT 10;

-- UPDATE
UPDATE leads SET status = 'contacted' WHERE id = 42;

-- DELETE
DELETE FROM leads WHERE id = 42;

-- UPSERT (الأهم عملياً في الأتمتة - (أدخل أو حدّث)
INSERT INTO leads (email, name) VALUES ('m@example.com', 'محمد')
ON CONFLICT (email) DO UPDATE SET name = EXCLUDED.name;
```

⚠️ **UPDATE** أو **DELETE** بلا **WHERE** يطال كل الصفوف. قاعدة السلامة: نفذ **SELECT** بنفس شرط **WHERE** أولاً للتأكد من عدد الصفوف المتأثرة، ثم بذل إلى **DELETE / UPDATE**. على قاعدة بيانات عميل، هذه الخطوة غير قابلة للتفاوض.

💡 **UPSERT** هو صديقك الأول في الأتمتة. يحل مشكلة "هل هذا السجل موجود؟" في عملية واحدة، ويمنع التكرار عند وصول webhook مرتين. عقدة Postgres (v2.7) توفره كعملية جاهزة بلا كتابة SQL.

9.3 تصميم المخطط

مثال — نظام عملاء محتملين 

```
CREATE TABLE leads (  
  id          BIGSERIAL PRIMARY KEY,  
  email       TEXT UNIQUE NOT NULL,      -- يمنع التكرار على مستوى القاعدة  
  phone       TEXT UNIQUE,  
  name        TEXT NOT NULL,  
  company     TEXT,  
  budget      DECIMAL(12,2),  
  source      TEXT NOT NULL,  
  status      TEXT NOT NULL DEFAULT 'new',  
  score       INTEGER DEFAULT 0,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE activities (  
  id          BIGSERIAL PRIMARY KEY,  
  lead_id     BIGINT NOT NULL REFERENCES leads(id) ON DELETE CASCADE,  
  type        TEXT NOT NULL,  
  content     TEXT,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_leads_status ON leads(status);  
CREATE INDEX idx_leads_email  ON leads(email);  
CREATE INDEX idx_act_lead     ON activities(lead_id);
```

لماذا هذه القرارات؟ 💡

القرار	السبب
email UNIQUE	الحماية الحقيقية من التكرار — على مستوى القاعدة لا على مستوى المنطق. حتى لو أخطأ ال workflow، القاعدة ترفض.
REFERENCES leads(id)	يضمن ألا يوجد نشاط بلا عميل (سلامة مرجعية)
ON DELETE CASCADE	حذف العميل يحذف أنشطته — لا بيانات يتيمة
INDEX على الأعمدة المستعلم بها	استعلامات أسرع بمراحل عند النمو
DEFAULT NOW()	لا تعتمد على n8n لضبط الوقت
NOT NULL	يمنع البيانات الناقصة من الدخول أصلاً

🧠 مبدأ معماري أساسي: ضع قيود السلامة في قاعدة البيانات، لا في ال workflow فقط. ال workflow قد يُعَدَّل أو يُتجاوز أو يُستدعى من مصدر آخر. قاعدة البيانات هي خط الدفاع الأخير، وهي التي لا تنسى.

⚠️ الفهارس ليست مجانية. تُسرّع القراءة وتُبطئ الكتابة قليلاً وتستهلك مساحة. فهرس الأعمدة التي تستعلم بها فعلاً (في WHERE و JOIN و ORDER BY)، لا كل الأعمدة.

9.4 العلاقات

النوع	مثال	التنفيذ
One-to-Many	عميل ← عدة أنشطة	مفتاح أجنبي في جدول "many"
Many-to-Many	طلبات ↔ منتجات	جدول وسيط
One-to-One	مستخدم ← ملف شخصي	مفتاح أجنبي فريد


```
-- Many-to-Many وسيط جدول
CREATE TABLE order_items (
  order_id BIGINT REFERENCES orders(id) ON DELETE CASCADE,
  product_id BIGINT REFERENCES products(id),
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
```

🔔 **unit_price** مخزن في جدول الطلب وليس مقروءًا من **products**. لماذا؟ لأن سعر المنتج يتغير، والطلب القديم يجب أن يحتفظ بالسعر وقت الشراء. هذا يُسمى لقطة زمنية (Point-in-time snapshot) — وهو خطأ شائع جدًا في التصميم يظهر أثره بعد أول تغيير أسعار، حين تتغير قيم الفواتير القديمة بأثر رجعي.

9.5 الأمان

الممارسة	لماذا
مستخدم مخصص بأقل صلاحية	لا تستخدم <code>postgres</code> الرئيسي في n8n
استعلامات معاملة (Parameterized)	تمنع SQL Injection — أخطر ثغرة
SSL للاتصال	تشفير البيانات أثناء النقل
قصر الوصول بعنوان IP	تقليل سطح الهجوم
نسخ احتياطية دورية مُختبرة	نسخة لم تُختبر استعادتها ليست نسخة

🔒 SQL Injection — الشرح والخطر:

خطأ قاتل (بناء الاستعلام بالدمج النمي):

```
SELECT * FROM users WHERE email = '{{ $json.email }}'
```

إن أرسل مهاجم في الحقل: `-- ' OR '1'='1'` يصبح الاستعلام:

```
SELECT * FROM users WHERE email = '' OR '1'='1' --'
```

والنتيجة: كل سجلات الجدول تُعاد. وبأسلوب مشابه يمكن حذف جداول كاملة.

الصواب: استخدم **Query Parameters** في عقدة Postgres (\$1 , \$2) وضع القيم في قائمة المعاملات المنفصلة. بهذه الطريقة تُعامل القيم كبيانات دائمة، ولا تُفسر كأوامر SQL مهما كان محتواها.

هذه ليست حالة نظرية. أي حقل يأتي من webhook أو نموذج أو رسالة عميل هو مدخل غير موثوق.

Common Mistakes — Module 9 ⚠️

الخطأ	التصحيح
FLOAT للنقود	DECIMAL
Timestamp بلا منطقة زمنية	TIMESTAMPTZ
بلا قيد UNIQUE	أضفه على الحقول المميزة
DELETE / UPDATE بلا WHERE	اختبر بـ SELECT أولاً
دمج نصي في SQL	معاملات \$1
بلا فهرس	افهرس أعمدة JOIN / WHERE
قراءة السعر الحالي للطلب القديم	خزن السعر وقت الشراء
مستخدم قاعدة بيانات بصلاحيات كاملة	أقل صلاحية ممكنة

Mini Quiz — Module 9 📝

1. لماذا DECIMAL لا FLOAT للمبالغ؟
2. ما UPSERT ولماذا هو محوري في الأتمتة؟
3. اشرح SQL Injection وكيف يمنعها n8n.
4. لماذا نخزن سعر المنتج في جدول الطلب؟
5. متى Data Table ومتى Postgres؟
6. ما فائدة ON DELETE CASCADE؟

الإجابات ▼

1. لأن FLOAT تقريبي وينتج أخطاء تراكمية في الحسابات المالية؛ DECIMAL دقيق عشريًا.
2. UPSERT = أدخل السجل، وإن كان موجودًا فحدّثه. محوري لأنه يحل "هل هذا موجود؟" بعملية واحدة ويمنع التكرار عند وصول نفس الحدث مرتين.
3. حقن أوامر SQL عبر مدخلات المستخدم لتغيير معنى الاستعلام (قراءة أو حذف بيانات). المنع: استعلامات معاملية \$1 / \$2 تُعامل القيم كبيانات لا كأوامر.
4. لأن السعر يتغير، والفاتورة القديمة يجب أن تحفظ سعر وقت الشراء — وإلا تغيرت قيم الطلبات التاريخية بأثر رجعي.
5. Data Table للنماذج الأولية والبيانات البسيطة بلا بنية خارجية. Postgres للأنظمة الإنتاجية ذات العلاقات والنمو والاستعلامات المعقدة.
6. يضمن حذف السجلات التابعة تلقائيًا عند حذف الأصل، فلا تبقى بيانات يتيمة تشير إلى شيء غير موجود.

Definition of Done — Module 9 ✓

- ☐ صَمِّمْتُ مخططًا بجدولين مرتبطين وقيود صحيحة.
 - ☐ نَقَذْتُ CRUD كاملاً من n8n.
 - ☐ استخدمْتُ UPSERT لمنع التكرار.
 - ☐ أستخدم استعلامات معاملة دائماً.
 - ☐ أضفتُ فهرس مبررة.
 - ☐ أختبر WHERE بـ SELECT قبل DELETE / UPDATE.
-

المستوى 4 — AI & Resilience

Module 10 — AI Automation

الهدف: بناء أنظمة ذكية موثوقة — لا عروض تقنية هشة. المدة: 240 دقيقة · المتطلبات: Modules 4-9

كل الإصدارات مُتحققة من نسخة n8n حقيقية.

10.1 كيف تعمل نماذج اللغة؟

الفكرة ببساطة 💡

ال LLM هو مُكمّل نص فائق القدرة. تعطيه نصًا، فيتوقع ما يجب أن يليه بناءً على أنماط تعلّمها.

النتائج العملية لهذه الحقيقة:

الخاصية	الأثر عليك
احتمالي لا حتمي	نفس السؤال قد يعطي إجابتين مختلفتين
بلا ذاكرة افتراضية	كل استدعاء يبدأ من الصفر — الذاكرة تُبني يدويًا
يهلوس بثقة	يخترع معلومات بأسلوب واثق تمامًا
حدود السياق	لا يقبل نصًا لا نهائيًا
مكلف نسبيًا	تدفع بال tokens (المدخل والمخرج)

⚠️ "يهلوس بثقة" هو أخطر ما في القائمة. النموذج لا يقول "لا أعرف" — بل يخترع إجابة تبدو صحيحة تمامًا. لا يوجد مؤشر خطأ. في أنظمة العملاء، هذه ليست مشكلة نظرية. نموذج يصنّف عميلًا غاضبًا كـ "مهتم" يكلف صفقة حقيقية.

المصطلح	المعنى	الأثر العملي
Token	جزء من كلمة	وحدة التسعير — العربية تستهلك tokens أكثر من الإنجليزية
Context Window	أقصى نص يستوعبه	يحدد كم تاريخ محادثة تمرر
Temperature	العشوائية (0-1)	0 للتصنيف +0.7 للإبداع
System Prompt	تعليمات الدور الثابتة	مكان القواعد والقيود
Structured Output	إجبار المخرج على صيغة	إلزامي في الأتمتة

★ **Temperature = 0** للتصنيف والاستخراج. هذا يقلل التذبذب ويجعل النظام قابلاً للاختبار. استخدام الافتراضي (عادةً أعلى) في مهمة تصنيف يعني نتائج مختلفة لنفس المدخل — كابوس تشخيصي.

10.2 خريطة عقد الذكاء الاصطناعي

📌 مُتحقَّقة من نسخة حقيقية:

المعدة	typeVersion	الاستخدام
AI Agent	@n8n/n8n-nodes-langchain.agent v3.1	وكيل يخطط ويستخدم أدوات
Text Classifier	v1.1	تصنيف لفئات — مخرج منفصل لكل فئة
Information Extractor	v1.2	استخراج حقول منظّمة من نص
Structured Output Parser	v1.3	إجبار المخرج على مخطط JSON
Simple Memory	memoryBufferWindow v1.4	ذاكرة محادثة بلا credentials
Postgres Chat Memory	v1.4	ذاكرة دائمة في قاعدة بيانات
PGVector Store	v1.3	قاعدة متجهية على Postgres
Embeddings OpenAI	v1.2	تحويل النص لمتجهات
OpenAI Chat Model	lmChatOpenAi v1.3	نموذج يُوصَل بالوكيل
AI Agent Tool	agentTool v3	وكيل كأداة لوكيل آخر
Recursive Character Text Splitter	v1	تقسيم النصوص الطويلة

🔗 Gateway Credits على n8n Cloud: تتيح استهلاك نماذج (Gemini · OpenAI · Anthropic وغيرها) وخدمات (Firecrawl · Brave) (Search · PDF.co) بلا إعداد مفاتيح — الفوترة بالاستخدام. ممتاز للتعلم والنماذج الأولية، للإنتاج طويل الأمد، قارن التكلفة بالمفاتيح المباشرة.

10.3 Structured Output — الفارق بين عرض تقني ونظام

⚠️ المشكلة

اطلب من LLM تصنيف رسالة، فقد يجيب:

"أعتقد أن هذا العميل مهتم بالشراء، يبدو أن ميزانيته جيدة"

نص جميل، لا يمكن لأي عقدة استخدامه. لا يمكن بناء IF عليه، ولا حفظه في عمود.

✓ الحل

Structured Output Parser يجبر المخرج على مخطط محدد:

```
{
  "intent": "PURCHASE_INTENT",
  "sentiment": "positive",
  "budget_signal": "high",
  "needs_human": false,
  "confidence": 0.87
}
```

الآن يمكن: بناء IF على needs_human · حفظ intent في عمود · التوجيه بـ Switch حسب sentiment .

⚠️ ملاحظة تقنية مُتحققة وحاسمة: مخرج Structured Output Parser مغلف داخل مفتاح output .

```
{ "output": { "intent": "PURCHASE_INTENT", "confidence": 0.87 } }
```

لِلوصول: {{ \$json.output.intent }} — وليس {{ \$json.intent }}

هذا سبب متكرر جدًا لـ undefined بعد عقدة AI، ويُهدر وقتًا طويلاً لمن لا يعرفه.

🧠 القاعدة الذهبية: كل استدعاء LLM في نظام إنتاجي يجب أن يُنتج مخرجًا منظمًا. النص الحر مقبول فقط حين يكون المخرج النهائي موجّهًا لإنسان (كتابة رسالة مثلاً).

10.4 هندسة الأوامر (Prompting) للأتمتة

الأوامر في الأتمتة تختلف عن المحادثة: تحتاج ثباتًا لا إبداعًا.

⚙️ هيكل الأمر الإنتاجي — ستة أقسام

- الدور. أنت مساعد تصنيف رسائل العملاء لشركة برمجيات.
- المهمة. صنف الرسالة إلى فئة واحدة من الفئات المحددة.
- الفئات. INTERESTED | PRICE_QUESTION | COMPLAINT | SPAM | OTHER
- القواعد - اختر فئة واحدة فقط.
 - OTHER إن كان النص غامضًا استخدم -
 - لا تخترع فئات جديدة -
 - needs_human = true إن كان العميل غامضًا اضبط -
- "كم السعر؟" → PRICE_QUESTION أمثلة
- "!الخدمة سيئة" → COMPLAINT (needs_human = true)
- صيغة المخرج (Structured Output Parser يُفرض عبر)

★ Best Practices

الممارسة	لماذا
كن محددًا في الفئات	"صنف الرسالة" غامض؛ عدّد الخيارات صراحةً
أضف فئة OTHER	يمنع إجبار النموذج على تصنيف خاطئ
أعط أمثلة (Few-shot)	يرفع الدقة أكثر من أي تحسين آخر
حدد ما لا يفعله	"لا تخترع فئات"
اطلب مؤشر ثقة	يتيح توجيه الحالات الضعيفة لمراجعة بشرية
Temperature = 0	ثبات النتائج

🚀 نمط إنتاجي — بوابة الثقة:

AI Agent → IF (confidence > 0.8)

- └ true → معالجة تلقائية
- └ false → مراجعة بشرية

هذا النمط وحده يرفع موثوقية أي نظام AI بشكل جذري، ويجعله قابلاً للتسليم لعميل حقيقي. بدون، النظام يتصرف بثقة كاملة في الحالات التي لا يفهمها — وهذا بالضبط ما يفشل أمام العميل.

الفرق بين استدعاء LLM ووكيل

القدرة	استدعاء LLM	AI Agent
يولّد نصًا	يخطط ويستخدم أدوات ويكرر	
الأدوات	لا	نعم (بحث، قاعدة بيانات، API)
الذاكرة	لا	نعم
الخطوات	واحدة	متعددة حسب الحاجة
التكلفة	منخفضة	أعلى بكثير (استدعاءات متعددة)
القابلية للتنبؤ	أعلى	أقل

مكونات الوكيل



المكوّن	الدور	ملاحظة
Model	العقل	إلزامي
Memory	تذكّر المحادثة	Simple Memory للبداية · Postgres للإنتاج
Tools	القدرة على الفعل	قاعدة بيانات، بحث، HTTP، وكيل آخر
Output Parser	ضبط شكل المخرج	مهم للأتمتة

⚠️ ملاحظة مُتحقّقة: داخل العقد الفرعية (subnodes) للوكيل، لا تعتمد على **\$json** — العقد الفرعية لا تشارك سياق الـ item الخاص بالعقدة السابقة. استخدم مرجعًا صريحًا لعقدة المشغل. هذا سبب شائع لظهور قيم فارغة داخل إعدادات الأدوات والذاكرة.

⚠️ الذاكرة مشتركة عبر الجلسات إن لم تعزلها. اضبط **Session ID** بمعرف المستخدم (رقم هاتف، معرف محادثة). بدونها ستختلط محادثات العملاء ببعضها — تسريب بيانات بين العملاء، وهي مشكلة أمنية لا مجرد خطأ منطقي.

معظم مهام الأتمتة لا تحتاج وكيلًا.

الحاجة	الأداة الصحيحة
تصنيف نص	Text Classifier (أرخص، أسرع، أدق)
استخراج حقول	Information Extractor
توليد نص بـ قالب	استدعاء LLM بسيط
خطوات متعددة غير معروفة مسبقًا	AI Agent ✓

استخدام وكيل لمهمة تصنيف بسيطة = دفع عشرة أضعاف التكلفة مقابل نتيجة أقل تنبؤًا. هذا خطأ شائع جدًا لدى من يتحمس للوكلاء.

10.6 RAG — التوليد المعزز بالاسترجاع

؟ المشكلة

النموذج لا يعرف بيانات شركتك: أسعارك، سياساتك، منتجاتك.

الحلول الثلاثة ⚙️

الحل	الوصف	متى
1. في الأمر مباشرة	ألصق المعلومات في الـ prompt	معلومات قليلة وثابتة
2. RAG	ابحث عن المقاطع ذات الصلة ومررها	الأشيع والأنسب
3. Fine-tuning	درّب النموذج	مكلف · نادرًا ما يلزم

كيف يعمل RAG؟ 💡

مرحلة الفهرسة (مرة واحدة، وتكرر عند تحديث المحتوى):
تخزين في قاعدة متجهية → (Embeddings) مستندات → تقسيم لمقاطع → تحويل لمتجهات

مرحلة الاستعلام (كل سؤال):
مقاطع → تُمرّر للنموذج مع السؤال → إجابة N سؤال → تحويل لمتجه → بحث تشابه → أفضل

ما هي الـ Embeddings؟ 💡

تحويل النص إلى قائمة أرقام تمثل معناه. النصوص المتشابهة معني تكون متجهاتها متقاربة رياضيًا.

النتيجة العملية: البحث عن "كم تكلفة الخدمة؟" يجد مقطعًا مكتوبًا فيه "أسعارنا تبدأ من..." — رغم عدم تطابق أي كلمة. هذا هو البحث الدلالي، والفرق بينه وبين البحث النقي هو جوهر قيمة RAG.

⚙️ أوضاع Vector Store (مُتحَقَّقة)

الوضع	الاستخدام
insert	إدخال المستندات (مرحلة الفهرسة)
load	بحث تشابه لمرة واحدة في المسار الرئيسي
retrieve-as-tool	الوضع المعياري لـ RAG — يُوصَل كأداة للوكيل
retrieve	يُعرض كعقدة فرعية لعقدة أخرى

★ RAG Best Practices

الممارسة	لماذا
حجم مقطع 300-800 كلمة	صغير جدًا يفقد السياق · كبير جدًا يخفف الصلة
تداخل بين المقاطع (~10-15%)	يمنع قطع المعنى عند الحدود
أضف بيانات وصفية	المصدر والقسم — للاستشهاد والتصفية
اختبر الاسترجاع منفصلاً	قبل توصيله بالنموذج
اطلب الاستشهاد بالمصدر	يقلل الهلوسة و يتيح التحقق
أعد الفهرسة عند تغيير المحتوى	وإلا أجاب بمعلومات قديمة

🔍 تشخيص RAG — القاعدة الأهم: عندما تكون الإجابات سيئة، 90% من الوقت المشكلة في الاسترجاع لا في النموذج.

اختبر الاسترجاع وحده: استخدم وضع load وافحص المقاطع المُسترجَعة.

- هل هي ذات صلة فعلياً بالسؤال؟ ← إن لا، فالمشكلة في التقسيم أو الفهرسة أو الـ embeddings.
- هل تحتوي الإجابة أصلاً؟ ← إن لا، فالمحتوى غير مفهرس أو مقطوع.

لا تعُد الـ prompt قبل التحقق من الاسترجاع. هذا أشهر مضيعة للوقت في مشاريع RAG.

10.7 التدخل البشري (Human-in-the-Loop)

💡 الفكرة

النظام يتوقف وينتظر قرار إنسان قبل المتابعة.

🚀 **مُتَحَقِّق:** عمليات `sendAndWait` متاحة في عقد متعددة: [Telegram](#) · [Gmail](#) · [Discord](#) · [Google Chat](#) · [WhatsApp](#) · [Send Email](#) — بالإضافة إلى عقد HITL مخصصة كأدوات للوكلاء.

🚀 متى تستخدمه؟

الحالة	لماذا
إرسال رسالة باسم العميل	خطأ واحد يضر سمعته
قرار مالي	لا رجعة عنه
ثقة النموذج منخفضة	مراجعة بدل تخمين
بيانات حساسة	مسؤولية قانونية
أول أسبوعين من أي نظام AI جديد	بناء الثقة تدريجيًا

🚀 استراتيجية تسليم للعملاء — التدرّج في الأتمتة:

المرحلة	النمط	المدة
1	AI يقترح، الإنسان يعتمد كل شيء	أسبوعان
2	اعتماد تلقائي عند ثقة > 0.9 ، والباقي يدوي	شهر
3	تلقائي بالكامل + عيّنة مراجعة عشوائية	مستمر

لماذا هذا التدرج مهم تجاريًا؟ العميل الذي يرى AI يرسل رسالة خاطئة في اليوم الأول يفقد الثقة في النظام كله ولا يستعيدها. التدرّج يبني الثقة ويعطيك بيانات حقيقية لضبط النظام. هذه ممارسة تسليم، لا مجرد ممارسة تقنية.

10.8 التحكم في التكلفة

الأسلوب	التوفير
نموذج أصغر للمهام البسيطة	التصنيف لا يحتاج أقوى نموذج
تصفية قبل الاستدعاء	لا ترسل الرسائل غير المؤهلة أصلاً
تخزين مؤقت للناتج المتكررة	نفس السؤال = نفس الإجابة
تقليص السياق	لا ترسل كل التاريخ في كل مرة
maxTokens محدود	يمنع مخرجات طويلة بلا داع
Pin Data أثناء التطوير	لا تدفع مقابل كل تجربة

⚠ احسب التكلفة لكل تنفيذ قبل التسليم. نظام يكلف 0.02 دولار لكل رسالة يبدو رخيصةً — حتى يصل 10,000 رسالة شهرياً = 200 دولار. اعرف الرقم قبل أن يعرفه العميل. ولا تسعّر مشروعًا بتكلفة تشغيل مجهولة.

Common Mistakes — Module 10 ⚠

الخطأ	الأثر	التصحيح
مخرج نمي حر	لا يمكن استخدامه آلياً	Structured Output
نسيان <code>\$json.output.x</code>	undefined دائم	المخرج مغلف في <code>output</code>
بلا Session ID للذاكرة	اختلاط محادثات العملاء	اضبط معرّف الجلسة
وكيل لمهمة تصنيف	تكلفة عالية ودقة أقل	Text Classifier
Temperature عالية للتصنيف	نتائج متذبذبة	0
بلا مسار احتياطي	انهيار عند فشل التحليل	Fallback + تنبيه
ضبط الـ prompt قبل فحص الاسترجاع	إهدار وقت	افحص RAG أولاً
تجاهل التكلفة	مفاجأة في الفاتورة	احسب قبل التسليم
ثقة عمياء بالمخرج	أخطاء صامتة	بوابة ثقة + مراجعة

Mini Quiz — Module 10 🖋

1. لماذا Structured Output إلزامي في الأتمتة؟

2. أين تجد مخرج Structured Output Parser بالضبط؟

3. متى Text Classifier ومتى AI Agent؟

4. ماذا يحدث بلا Session ID للذاكرة؟ ولماذا هي مشكلة أمنية؟

5. إجابات RAG سيئة — ما أول ما تفحصه ولماذا؟

6. اشرح نمط بوابة الثقة.

الإجابات ▼

1. لأن المخرج النمي الحر لا يمكن للعقد التالية استخدامه: لا IF ولا حفظ في عمود ولا توجيه. المخرج المنظم يجعل نتيجة النموذج بيانات قابلة للمعالجة.

2. داخل مفتاح output : {{ \$json.output.fieldName }} — لا {{ \$json.fieldName }} .

3. Text Classifier للتصنيف لفئات معروفة (أرخص وأسرع وأدق). AI Agent حين تكون الخطوات متعددة وغير معروفة مسبقًا وتحتاج أدوات.

4. تختلط محادثات كل المستخدمين في ذاكرة واحدة، فيرى النموذج سياق عميل آخر — تسريب بيانات بين العملاء، وهي مشكلة خصوصية لا مجرد خطأ منطقي.

5. الاسترجاع، لا النموذج. افحص المقاطع المُسترجعة بوضع load : هل هي ذات صلة؟ هل تحوي الإجابة أصلًا؟ 90% من مشاكل RAG سببها الاسترجاع.

6. توجيه المخرجات ذات الثقة العالية للمعالجة التلقائية، والمنخفضة لمراجعة بشرية — بدل تصرف النظام بثقة كاملة في حالات لا يفهمها.

Definition of Done — Module 10 ✓

- ☐ بنيت استدعاء AI بمخرج منظم واستخدمت حقوله في عقد لاحقة.
- ☐ أعرف أن المخرج داخل output .
- ☐ بنيت AI Agent بنموذج وذاكرة وأداة واحدة على الأقل.
- ☐ ضبطت Session ID لعزل المحادثات.
- ☐ بنيت RAG بسيطًا واختبرت الاسترجاع منفصلًا.
- ☐ أضفت بوابة ثقة + مسارًا للمراجعة البشرية.
- ☐ حسبت تكلفة التنفيذ الواحد.

Module 11 — Error Handling

الهدف: بناء أنظمة تتوقع الفشل بدل أن تتفاجأ به. المدة: 150 دقيقة · المتطلبات: Modules 6, 7

11.1 لماذا تفشل الـ Workflows؟

🧠 التحول الذهني الأهم في هذا المستوى: الفشل ليس استثناءً. الفشل هو الحالة الطبيعية على مدى زمني كافٍ. API سيتوقف. شبكة ستقطع. صلاحية ستنتهي. حقل سيصل فارغاً. عميل سيرسل بيانات غريبة. السؤال ليس "هل سيفشل؟" بل "ماذا يحدث حين يفشل؟"

⚙️ تصنيف الأخطاء — وأثره على الاستراتيجية

الصفة	أمثلة	الاستراتيجية
عابر (Transient)	500 · انقطاع شبكة · timeout	أعد المحاولة — ينجح غالباً
دائم (Permanent)	401 · 404 · بيانات خاطئة	لا تُعد المحاولة — أبلغ وتوقف
حدّي (Throttling)	429	أبطئ ثم أعد
منطقي (Logic)	حقل مفقود · نوع خاطئ	تحقق مبكراً ووجه لمسار خطأ
بشري (Human)	إعدادات خاطئة	اختبر قبل الإنتاج

⚠️ إعادة المحاولة على خطأ دائم إهدار ضار. 401 لن يُصلحه التكرار. عشر محاولات = عشرة أخطاء + تأخير + احتمال حظر. افحص نوع الخطأ ثم قرر — لا تفعل Retry بشكل أعمى على كل شيء.

11.2 أدوات n8n لمعالجة الأخطاء

⚙️ الأدوات الأربع

الأداة	الموقع	الاستخدام
Retry On Fail	إعدادات العقدة	الأخطاء العابرة
Continue On Fail	إعدادات العقدة	متابعة رغم فشل item
Error Trigger	عقدة مشغلة (v1)	التقاط أخطاء workflow آخر
Error Workflow	إعدادات الـ workflow	ربط workflow معالجة الأخطاء

الإعداد	التوصية
Max Tries	5-3
Wait Between Tries	1000ms فأكثر، متزايد إن أمكن

⚠️ انتبه على POST: إعادة المحاولة قد تُنشئ سجلات مكررة (راجع §7.2). الحل: استخدم مفتاح Idempotency أو UPSERT بدل INSERT.

Continue On Fail ⚙️ — أداة قوية وخطيرة

تجعل الـ workflow يواصل رغم فشل عقدة.

الوضع	متى
Continue (regular output)	الفشل مقبول والبيانات تكمل
Continue (using error output)	👉 الأفضل — مسار منفصل للفشل
Stop	الفشل يجب أن يوقف كل شيء

⚠️ الاستخدام الخاطئ لـ Continue On Fail يخفي المشاكل. إن واصلت دون تسجيل الفشل، سيبدو النظام ناجحًا وهو يفقد بيانات صامتًا — وهذا أسوأ من الفشل الصريح، لأنك لا تكتشفه.
القاعدة: كلما فُعلت Continue On Fail، يجب أن يوجد مسار يسجل الفشل وينبّه عليه.

Error Workflow ⚙️ — شبكة الأمان المركزية

```
[Error Trigger]
|
├─ للتحليل لاحقًا ← [تسجيل في قاعدة البيانات]
├─ Telegram / Email ← [إشعار الفريق]
└─ [تنبيه عاجل] → [IF: ؟ حرج]
```

البيانات المتاحة في Error Trigger: اسم الـ workflow · العقدة الفاشلة · رسالة الخطأ · معرف التنفيذ · البيانات وقت الفشل.

★ أنشئ Error Workflow واحدًا واربطه بكل workflows الإنتاج. هذا أعلى عائد لأقل جهد في المنهج كله: نصف ساعة عمل تعطيك رؤية كاملة على كل أنظمتك. بدون، تكتشف الأعطال من العميل — وهذا أسوأ سيناريو مهني ممكن.

11.3 التحقق من المدخلات (Validation)

🧠 **تحقق مبكراً، وافشل بوضوح.** اكتشاف حقل مفقود في العقدة الأولى أرخص بكثير من اكتشافه في العقدة العشرين بعد كتابة بيانات ناقصة في قاعدة البيانات.

🔧 نمط بوابة التحقق

```
[Webhook]
|
▼
الشرط: كل الحقول المطلوبة موجودة وصحيحة الشكل - [التحقق: IF]
|
├ true → [المعالجة الطبيعية]
|
└ false → [رسالة واضحة Respond 400] → [سجل الرفض]
```

ما الذي نتحقق منه؟

الفحص	مثال
الوجود	هل email موجود وغير فارغ؟
النوع	هل budget رقم فعلاً؟
المصيغة	هل البريد يطابق نمط بريد صحيح؟
المدى	هل score بين 0 و 100؟
قائمة مسموحة	هل status من القيم المعروفة؟

🔒 **التحقق ممارسة أمنية أيضاً.** المدخل غير الموثوق هو مصدر SQL Injection وتلوث البيانات. لا تثق ببيانات لم تولدها أنت — أبداً.

11.4 التكرار و Idempotency

⚠️ **المشكلة الصامتة**

Webhooks تصل مرتين. هذا ليس عطلاً — بل سلوك متوقع:

- المزود أعاد المحاولة لأنه لم يستلم ردًا في الوقت المحدد
- انقطاع شبكة أثناء الرد
- المستخدم ضغط الزر مرتين

النتيجة بلا حماية: عميلان بنفس البيانات · رسالتان للعميل · خصمان ماليان.

الحل	التنفيذ	القوة
1. Remove Duplicates	عملية <code>removeItemsSeenInPreviousExecutions</code>	✓ الأسهل — جاهز بلا بنية
2. UPSERT	ON CONFLICT DO UPDATE في قاعدة البيانات	✓ قوي
3. UNIQUE قيد	على مستوى مخطط القاعدة	✓✓ الأقوى — لا يمكن تجاوزه

💡 الحل رقم 1 كنز مهمّل. كثيرون يبنون جداول تتبع معقدة، بينما عقدة **Remove Duplicates** توفر هذا جاهزًا. الأفضل عمليًا: اجمع بين 1 و 3 — دفاع في العمق. المنطق يمنع، والقاعدة تضمن.

الفكرة الجوهرية 🧠

Idempotent = تنفيذ العملية مرة أو عشر مرات يُنتج نفس الحالة النهائية.
 INSERT غير idempotent (ينشئ نسخًا). UPSERT idempotent. صمّم عملياتك لتكون **idempotent**. فتصبح إعادة المحاولة آمنة تمامًا — وهذا يفتح لك باب Retry بلا قلق.

11.5 المراقبة والتنبيه

⚠️ السؤال الذي يفصل المحترف

"كيف أعرف أن نظامي توقف؟"
 إن كانت الإجابة "سيخبرني العميل" — فأنت لا تملك نظام مراقبة، وستفقد ذلك العميل.

⚙️ ثلاث طبقات مراقبة

الطبقة	ماذا تراقب	التنفيذ
1. أخطاء التنفيذ	فشل صريح	Error Workflow
2. الصمت	لم يعمل النظام إطلاقًا	Heartbeat
3. المنطق	يعمل لكن بنتائج خاطئة	فحوص دورية

🔑 **الطبعة 2** هي التي ينساها الجميع — وهي الأخطر. إن توقف الـ Trigger عن العمل، لن يقع أي خطأ — لأن لا شيء يعمل أصلاً. الأخطاء لا تُنتج إشعاراً حين لا يوجد تنفيذ.

Heartbeat: workflow مجدول يفحص: "هل عاجنا شيئاً في آخر X ساعة؟" إن لا → تنبيه. هذا نمط بسيط يكشف الأعطال الصامتة التي لا يكشفها أي شيء آخر.

★ ماذا تسجل؟

لا تسجل	سجل
🔑 كلمات المرور والمفاتيح	معرف التنفيذ ووقته
🔑 بيانات شخصية كاملة بلا داع	اسم الـ workflow والعقدة
🔑 أرقام بطاقات	رسالة الخطأ
🔑 محتوى رسائل حساسة	معرف السجل المتأثر
	المدة الزمنية

🔑 **السجلات** تُقرأ من أشخاص وتُخزن طويلاً. لا تضع فيها ما لا تريد تسريبه.

Debugging Lab 🐛 — مقدمة

تدريب التشخيص الكامل في: `exercises/DEBUGGING_LAB.md` يحتوي 12 تمريناً بأخطاء متعمدة مع نظام تلميحات متدرج.

منهجية التشخيص السبعية — احفظها:

#	السؤال
1	ما الأعراض بالضبط؟ (لا "لا يعمل")
2	أين حدث؟ (أي عقدة، أي تنفيذ)
3	ما الذي دخل تلك العقدة فعلاً؟ (افحص، لا تفترض)
4	ما الذي توقعته العقدة؟
5	ما الفجوة بين 3 و 4؟ ← هذا هو السبب
6	ما الحل السريع؟ وما الحل الجذري؟
7	كيف أمتنع تكراره؟ (تحقق · اختبار · مراقبة)

🧠 الفرق بين مبتدئ ومحترف في التشخيص: المبتدئ يبدأ من الحل ("أضيف Retry؟ أغير العقدة؟") ويجرب عشوائيًا. المحترف يبدأ من الملاحظة ("ما الذي دخل؟ ما الذي خرج؟") ويصل للسبب.

من يسأل AI فورًا عند أول خطأ يتخطى الخطوات 1-5 — فيحصل على حل قد ينجح، لكنه لا يبني القدرة على التشخيص المستقل. الاستخدام الصحيح لـ AI: نفذ الخطوات 1-5، ثم اسأل سؤالاً دقيقاً: "العقدة X استقبلت Y وتوقعت Z، لماذا؟" — هذا سؤال محترف يعطي إجابة دقيقة.

Common Mistakes — Module 11 ⚠️

الخطأ	التصحيح
Retry على خطأ دائم	صنف الخطأ أولاً
Continue On Fail بلا تسجيل	يخفي المشاكل — أضف مسار تسجيل
بلا Error Workflow	أنشئ واحدًا واربطه بالكل
بلا تحقق من المدخلات	بوابة تحقق بعد كل مصدر خارجي
تجاهل التكرار	UNIQUE + Remove Duplicates + قيد
بلا Heartbeat	الأعطال الصامتة لا تُكتشف
أسرار في السجلات	نقح ما تسجله

Mini Quiz — Module 11 🖋️

1. متى تُعيد المحاولة ومتى لا؟ أعط مثالاً لكل حالة.
2. لماذا Continue On Fail بلا تسجيل خطر؟
3. اذكر ثلاث طرق لمنع المعالجة المكررة.
4. لماذا لا يكفي Error Workflow وحده؟
5. ما معنى Idempotent وأيهما كذلك: INSERT أم UPSERT؟

📌 الإجابات ▼

1. أعد عند الأخطاء العابرة (500, timeout, انقطاع شبكة). لا تُعد عند الدائمة (401 مفتاح خاطئ، 404، بيانات غير صالحة) — التكرار لن يغيّر النتيجة ويهدر الوقت وقد يؤدي للخطر.
2. لأنه يجعل النظام يبدو ناجحًا بينما يفقد بياناتًا صامتًا؛ الفشل غير المرئي أسوأ من الفشل الصريح لأنك لا تكتشفه.
3. (1) Remove Duplicates بعملية removeItemsSeenInPreviousExecutions (2) UPSERT بدل (3) INSERT قيد UNIQUE في مخطط القاعدة.
4. لأنه يلتقط الأخطاء فقط. إن توقف الـ Trigger تمامًا فلا يقع خطأ أصلاً ولا يُنتج إشعارًا — تحتاج Heartbeat لكشف الصمت.
5. Idempotent = تكرار العملية يُنتج نفس الحالة النهائية. INSERT غير idempotent (ينشئ نسخًا مكررة)؛ UPSERT idempotent.

Definition of Done — Module 11 ✓

- ☐ أنشأت Error Workflow وربطته بـ workflow إنتاجي.
- ☐ أضفت بوابة تحقق بعد كل مصدر خارجي.
- ☐ نفذت حماية من التكرار بطريقتين على الأقل.
- ☐ أعرف متى أعيد المحاولة ومتى لا.
- ☐ بنيت Heartbeat يكشف الصمت.
- ☐ أشخص خطأ بالمنهجية السبعية دون سؤال AI.
- ☐ أنهيت 6 تمارين على الأقل من Debugging Lab.

Module 12 — Security

الهدف: حماية النظام والبيانات — ومسؤوليتك القانونية تجاه عملائك. المدة: 120 دقيقة · المتطلبات: Modules 7, 9

12.1 نموذج التهديد

اسأل قبل التأمين: ما الذي يمكن أن يُسرق أو يُخرب أو يُساء استخدامه؟

الأصل	التهديد	الأثر
Credentials	سرقة مفاتيح	وصول كامل لحسابات العميل
Webhook مفتوح	استدعاء غير مصرح	تلويث بيانات · استنزاف تكلفة AI
بيانات العملاء	تسريب	مسؤولية قانونية وسمعية
قاعدة البيانات	حقن SQL	سرقة أو حذف كامل
السجلات	أسرار مطبوعة	تسريب عبر قناة غير متوقعة

12.2 إدارة الأسرار

🔒 القواعد غير القابلة للتفاوض

✅ افعل	❌ لا تفعل
استخدم Credentials دائمًا	كتابة المفتاح في حفل عقدة
Environment Variables للإعدادات	تثبيت القيم في الكود
مفتاح منفصل لكل بيئة	نفس المفتاح للتطوير والإنتاج
أقل صلاحية ممكنة	صلاحيات كاملة "لأنه أسهل"
تدوير دوري للمفاتيح	مفتاح واحد للأبد
إبطال فوري عند الشك	التأجيل

⚠️ إن كنت قد كتبت مفتاحًا في مكان خاطئ ولو مرة — اعتبره مسرّبًا وأبطله. "سأحذفه لاحقًا" لا تعمل: الملف مُدِر، أو حُفظ في نسخة احتياطية، أو دخل تاريخ Git للأبد. الإبطال دقيقتان. التسريب قد يكلف عميلًا.

🔒 تحذير خاص بمستودعات Git: قبل رفع أي شيء، تأكد أن .env في .gitignore. مفتاح دخل تاريخ Git يبقى فيه حتى بعد حذفه من الملف — يحتاج إعادة كتابة التاريخ كاملاً.

12.3 تأمين الـ Webhooks

⚠️ Webhook بلا مصادقة = نقطة دخول مفتوحة للإنترنت كله. أي شخص يعرف الرابط يستطيع: إغراق قاعدة بياناتك · استنزاف رصيد AI لديك · إرسال رسائل باسم عميلك. الرابط ليس سرًا: يظهر في سجلات الوسطاء، وقد يُشارك في تذكرة دعم، أو يُكتب في وثيقة.

الطبقة	التنفيذ	القوة
1. مسار غير قابل للتخمين	UUID عشوائي بدل <code>/webhook/lead</code>	ضعيفة وحدها
2. Header Auth	رأس سرى طويل	✓ الحد الأدنى المقبول
3. Basic Auth	اسم وكلمة مرور	جيدة
4. التحقق من التوقيع (HMAC)	التحقق من توقيع المرسل	✓✓ الأقوى
5. تحقق من المحتوى	مخطط صارم للبيانات	✓ إلزامية دائمًا

💡 لماذا HMAC هو الأقوى؟

الخدمات الكبرى (Stripe، GitHub، Meta) ترسل توقيعًا في الـ header محسوبًا من محتوى الرسالة + سر مشترك. أنت تحسب التوقيع بنفسك وتقارنه. إن تطابق، فالرسالة من المرسل الحقيقي ولم تُعدّل في الطريق.

الميزة على Header Auth: حتى لو سُرّب الرابط، لا يستطيع المهاجم تزوير توقيع صحيح بلا معرفة السر.

★ الحد الأدنى المقبول لأي webhook إنتاجي: طبقة 2 + طبقة 5. إن كانت الخدمة تدعم HMAC، استخدمه — لا تكتفِ بالأقل حين يتوفر الأفضل.

12.4 التحقق والتنقية

الفحص	يمنع
قائمة مسموحة للحقول	حقول غير متوقعة تلوث البيانات
التحقق من النوع	أخطاء منطقية وتحولات خطيرة
حد أقصى للطول	إغراق التخزين
استعلامات معاملة	SQL Injection
ترميز المخرجات	XSS إن عُرضت البيانات في صفحة

🔑 قاعدة القوائم: استخدم قائمة مسموحة (Allowlist) لا قائمة ممنوعة (Blocklist). تعداد ما هو مسموح أكثر أمانًا من محاولة تعداد كل ما هو خطر — لأنك ستنسى شيئًا حتمًا.

12.5 الخصوصية وبيانات العملاء

المبدأ	التطبيق
تقليل البيانات	لا تجمع ما لا تحتاجه — أبسط حماية هي عدم الامتلاك
تحديد الغرض	استخدمها للغرض المعلن فقط
حد الاحتفاظ	احذف ما انتهت حاجته
ضبط الوصول	من يرى ماذا؟
التشفير	أثناء النقل وفي التخزين
حق الحذف	كيف تحذف بيانات شخص عند طلبه؟

⚠️ البيانات التي لا تجمعها لا يمكن تسريبها. قبل إضافة حق، اسأل: "هل نحتاجه فعلاً؟" كل حق إضافي مسؤولية إضافية.

12.6 ⚖️ الترخيص — قسم حرج لمن يبيع للعملاء

🔒 هذا القسم قد يوفر عليك مشكلة قانونية حقيقية. اقرأه كاملاً.

n8n يُنشر تحت Sustainable Use License — وهي ليست رخصة مفتوحة المصدر بالمعنى الكامل، بل **fair-code**.

🔗 مُتَحَقِّق من توثيق n8n ومركز المساعدة (سبتمبر 2026):

السيناريو	الحالة
استخدام n8n داخل عملك الخاص	✅ مسموح
بناء workflows لعميل على نسخته هو	✅ مسموح — الاستشارات مسموحة صراحةً
تقديم خدمات استشارية وبناء workflows	✅ مسموح بلا اتفاقية ترخيص منفصلة
استضافة workflows/credentials عملائك بشكل دائم على نسختك	⚠️ يتطلب ترخيصاً مدفوعاً
تضمين n8n داخل منتجك وإعادة بيعه	⚠️ يتطلب ترخيص Embed

🧠 ماذا يعني هذا لنموذج عملك؟

النموذج	الحالة	ملاحظة
أ. تبني على نسخة العميل ويدفع اشتراكه	✓ آمن تمامًا	النموذج الموصى به للبداية
ب. استشارات وبناء ثم تسليم	✓ آمن تمامًا	مسموح صراحةً
ج. تستضيف عملاءك على نسختك وتبيع اشتراكًا شهريًا	⚠ يتطلب ترخيصًا	راجع n8n قبل الإطلاق
د. تبني SaaS فوق n8n	⚠ يتطلب Embed	تواصل معهم

⚠ **النموذج (ج)** هو ما ينجرّف إليه كثيرون تدريجيًا دون انتباه: تبدأ ببناء نظام لعميل على نسختك "مؤقتًا للتجربة"، ثم يبقى هناك، ثم تضيف عميلًا ثانيًا وثالثًا — وفجأة تدير منصة استضافة تتطلب ترخيصًا.

النصيحة العملية: ابدأ بالنموذج (أ). العميل يفتح حسابه ويدفع اشتراكه، وأنت تتقاضى مقابل البناء والصيانة. مزايا إضافية: بيانات العميل عنده (مسؤولية أقل عليك) • لا تكاليف بنية تحتية تتحملها • النموذج قابل للتوسع بلا سقف.

إن أردت النموذج (ج) أو (د)، تواصل مع n8n للحصول على الترخيص المناسب قبل الإطلاق — لا بعده.

📌 هذا ملخص عملي لا استشارة قانونية. راجع نص الرخصة الرسمي واستشر مختصًا للحالات الكبيرة.

12.7 قائمة التحقق الأمني قبل الإنتاج

🔒 الأسرار

- [] لا مفاتيح مكتوبة في العقد
- [] Environment Variables أو Credentials كل الأسرار في
- [] مفاتيح منفصلة للتطوير والإنتاج
- [] أقل صلاحية ممكنة لكل مفتاح
- [] .gitignore. في .env

🔒 Webhooks

- [] (Header/Basic/HMAC) مصادقة مفعّلة
- [] مسار غير قابل للتخمين
- [] تحقق من المحتوى
- [] حد لحجم الطلب

🔒 قاعدة البيانات

- [] استعلامات معاملة فقط
- [] مستخدم بأقل صلاحية
- [] مفعّل SSL
- [] نسخ احتياطية مُختبرة

🔒 البيانات

- [] لا نجمع ما لا نحتاج
- [] لا أسرار في السجلات
- [] سياسة احتفاظ محددة
- [] آلية للحذف عند الطلب

🔒 الوصول

- [] n8n مراجعة من يملك وصولاً لـ
- [] مصادقة ثنائية مفعّلة
- [] بيئة إنتاج منفصلة عن التطوير

🔒 الترخيص

- [] نموذج العمل متوافق مع الرخصة

Mini Quiz — Module 12 📝

1. لماذا لا يكفي مسار webhook عشوائي كحماية؟
2. ما HMAC ولماذا أقوى من Header Auth؟
3. سَرِبَتْ مفتاحًا في اختبار سريع. ماذا تفعل؟
4. لماذا أفضل من Blocklist Allowlist؟
5. تريد استضافة workflows ثلاثة عملاء على نسختك مقابل اشتراك شهري. ما الحالة القانونية؟

1. لأن الرابط ليس سرًا حقيقيًا: يظهر في سجلات الوسيط وقد يُشارك في تذكرة دعم أو وثيقة. الغموض ليس أمانًا.
2. توقيع تشفيرى يُحسب من محتوى الرسالة + سر مشترك. أقوى لأنه يثبت هوية المرسل وسلامة المحتوى معًا؛ وحتى لو سُرّب الرابط لا يستطيع المهاجم تزوير توقيع صحيح.
3. أبطله فورًا وأنشئ بديلًا. لا تعتمد على حذفه لاحقًا — قد يكون مُدِر أو نُسخ احتياطيًا أو دخل تاريخ Git.
4. لأن تعداد المسموح محدود وقابل للضبط، بينما تعداد الممنوع لا ينتهي وستنسى حالة حتمًا.
5. يتطلب ترخيصًا مدفوعًا — استضافة workflows وcredentials العملاء بشكل دائم على نسختك خارج نطاق Sustainable Use License. البديل الآمن: كل عميل على نسخته وأنت تتقاضى مقابل البناء والصيانة.

Definition of Done — Module 12 ✓

- ☐ لا يوجد مفتاح واحد مكتوب في أي عقدة.
- ☐ كل webhooks الإنتاج مؤمنة.
- ☐ استعلامات معاملة في كل تعامل مع قاعدة البيانات.
- ☐ راجعت ما أسجله ونقيت الأسرار منه.
- ☐ أنهيت قائمة التحقق الأمني.
- ☐ أعرف موقع نموذج عملي من رخصة n8n.

المستوى 5 — Production & Architecture

Module 13 — Production Workflows

الهدف: الفرق بين "workflow يعمل" و "نظام جاهز للإنتاج". المدة: 120 دقيقة · المتطلبات: Modules 11, 12

13.1 التعريف

🧠 "يعمل" تعني: نجح مرة، على بيانات نظيفة، وأنت تراقبه. "جاهز للإنتاج" تعني: يعمل باستمرار، على بيانات فوضوية، وأنت نائم.

⚙️ الفروق السبعة

البُعد	يعمل	جاهز للإنتاج
البيانات	الحالة المثالية	ناقصة · خاطئة النوع · غير متوقعة
الفشل	ينهار	يتعافى أو يفشل بوضوح ويُبلغ
الحجم	10 سجلات	10,000 وأكثر
التكرار	يُنشئ نسخًا	idempotent
المراقبة	لا شيء	تنبيهات + سجلات + heartbeat
التوثيق	لا شيء	وصف + مخطط + دليل تشغيل
الصيانة	أنت فقط تفهمه	أي شخص يستطيع صيانته

✓ الموثوقية

- [] كل استدعاء خارجي له معالجة فشل
- [] مضبوط على الأخطاء العابرة فقط Retry
- [] مرتبط Error Workflow
- [] محددة Timeouts
- [] idempotent العمليات

✓ البيانات

- [] تحقق من كل مدخل خارجي
- [] التعامل مع القيم الفارغة والمفقودة
- [] أنواع البيانات مضبوطة صراحةً
- [] حماية من التكرار

✓ الأداء

- [] لا حلقات بلا داع
- [] محترمة Rate Limits
- [] مفعلة Pagination
- [] استعلامات مفهرسة
- [] لا تحميل كامل للبيانات في الذاكرة بلا داع

✓ المراقبة

- [] تنبيه عند الفشل
- [] يكشف الصمت Heartbeat
- [] سجلات كافية للتشخيص
- [] مؤشرات نجاح مُتَابَعَة

✓ الأمان

- [] كاملة Module 12 قائمة

✓ الصيانة

- [] أسماء واضحة لكل عقدة
- [] workflow وصف للـ
- [] عند القرارات غير البديهية Sticky Notes
- [] لا قيم مثبتة يجب أن تكون متغيرات
- [] منزوع Pin Data

✓ الاختبار

- [] المسار الناجح
- [] بيانات ناقصة
- [] بيانات خاطئة النوع
- [] فشل API
- [] Timeout
- [] طلب مكرر
- [] حجم كبير

13.3 اصطلاحات التسمية

العنصر	النمط	مثال
Workflow	[الوظيفة] - [النطاق]	Sales - Lead Intake
Sub-workflow	(Sub) [الوظيفة] - [النطاق]	Sales - Lead Scoring (Sub)
Node	فعل + مفعول	التحقق من البريد · حفظ العميل
Credential	[البيئة] - [الحساب] - [الخدمة]	عميل أ - إنتاج - Postgres
Error Workflow	Error Handler - [النطاق]	Sales - Error Handler

★ اختبار التسمية: افتح workflow بنيته قبل ثلاثة أشهر. إن احتجت أكثر من دقيقتين لفهم ما يفعله، فالتسمية والتوثيق فشلا. وإن كنت تسلم لعميل، فالاختبار أصعب: هل يستطيع مطور آخر صيانتَه بلا مساعدتك؟ إن لا، فأنت بعثَ نظامًا يحتجزه لديك — وهذا يضرّك أنت قبل العميل، لأنه يحوّلُك إلى دعم في مدى الحياة.

13.4 التوثيق

⚙ ما يجب أن يحتويه وصف كل workflow إنتاجي

الغرض: ماذا يفعل بجملة واحدة
المشغل: ما الذي يبدأه ومتى
المدخلات: شكل البيانات المتوقعة (مع مثال)
المخرجات: ماذا يُنتج وأين يذهب
التي يعتمد عليها workflows التبعيات: ما الخدمات/الـ
الأخطاء: كيف يتعامل مع الفشل
المالك: من المسؤول
آخر مراجعة: التاريخ

💡 الـ 17 workflow في حساب متقدم بلا وصف = 17 صندوقاً أسود. الوصف يستغرق دقيقتين ويوفّر ساعات لاحقاً — لك أو لمن يصون بعدك.

الممارسة	التنفيذ
بيئات منفصلة	تطوير · اختبار · إنتاج
سجل الإصدارات	n8n يحفظ تاريخ ال workflow — استخدمه
التصدير للنسخ الاحتياطي	صدّر JSON دوريًا (⚠️ بلا credentials)
الإصدار في الاسم	Sales - Lead Intake v2 عند التغييرات الكبرى
التراجع	اعرف كيف تعود للإصدار السابق قبل أن تحتاجه

⚠️ لا تعدّل على الإنتاج مباشرة. انسخ ال workflow → عدّل → اختبر → فعل الجديد → عطل القديم (لا تحذفه فورًا). الاحتفاظ بالقديم معطلًا لأسبوع يمنحك تراجعًا فوريًا عند اكتشاف مشكلة.

Definition of Done — Module 13 ✓

- ☐ حوّلت workflow قائمًا ليجتاز قائمة الجاهزية.
- ☐ طبّقت اصطلاحات التسمية.
- ☐ كتبت وصفًا كاملاً لكل workflow إنتاجي.
- ☐ اختبرت السيناريوهات السبعة.
- ☐ لدي خطة تراجع واضحة.

Module 14 — Deployment

الهدف: اختبار وتنفيذ بيئة تشغيل مناسبة. المدة: 180 دقيقة · المتطلبات: Module 13

14.1 مصفوفة اختيار البيئة

المعيار	n8n Cloud	Self-hosted (VPS)
الإعداد	دقائق	ساعات
الصيانة	عليهم	عليك
التكلفة	اشتراك ثابت	خادم (4-20\$) + وقتك
حدود التنفيذ	حسب الخطة	حسب موارد خادمك
البيانات	على خوادمهم	عندك بالكامل
Gateway Credits	✔ متاحة	✗
HTTPS	جاهز	تُعدّه بنفسك
النسخ الاحتياطي	مُدار	مسؤوليتك
التوسع	تغيير خطة	Queue Mode + Workers

🧠 قاعدة القرار:

- ابدأ بـ **Cloud** — تتعلم وتبني بلا إلهاء البنية التحتية.
 - انتقل لـ **Self-hosted** عندما: التكلفة تتجاوز الخادم • البيانات لا يجوز أن تغادر • تحتاج تحكماً كاملاً.
- 🚀 للعمل الحر: ابدأ كل عميل على حسابه هو في **Cloud**. أسباب متعددة تجتمع: تجاري (لا تتحمل تكاليف) • قانوني (متوافق مع الرخصة — راجع §12.6) • مسؤولية (بياناته عنده) • استمرارية (إن انتهت العلاقة، نظامه لا يتوقف). هذا القرار يبدو تفصيلاً إدارياً، لكنه في الحقيقة قرار معماري وقانوني وتجاري في آن واحد.

14.2 الاستضافة الذاتية بـ Docker

🔗 متغيرات البيئة أدناه مُتحقّقة من توثيق n8n.

المتغير	الغرض	ملاحظة حرجية
N8N_ENCRYPTION_KEY	تشفير الـ credentials	⚠️ احفظه في مكان آمن. فقدانه = فقدان كل الـ credentials نهائيًا
DB_TYPE=postgresdb	نوع قاعدة البيانات	⚠️ SQLite لا تصلح للإنتاج متعدد العمليات
DB_POSTGRESDB_HOST/PORT/DATABASE/USER/PASSWORD	اتصال القاعدة	
N8N_HOST	اسم النطاق	
N8N_PROTOCOL=https	البروتوكول	
WEBHOOK_URL	العنوان العام للـ webhooks	⚠️ بدون تكون روابط الـ webhooks خاطئة
GENERIC_TIMEZONE	المنطقة الزمنية	يمنع مشاكل الجدولة
N8N_BASIC_AUTH_ACTIVE	حماية الواجهة	فعّله دائمًا

⚠️⚠️ **N8N_ENCRYPTION_KEY** هو أخطر متغير على الإطلاق. يُستخدم لتشفير كل الـ credentials. إن فقدته، لن تستطيع فك تشفير أي **credential** — وستعيد إدخالها كلها يدويًا. عند الترقية أو النقل لخادم جديد، يجب استخدام نفس المفتاح. احفظه في مدير كلمات مرور، لا في الخادم وحده.

بنية إنتاجية بـ Docker Compose 🛠️



★ استخدم Caddy للبدائية — يحصل على شهادة HTTPS ويجدّها تلقائيًا بأقل إعداد ممكن.

#	الخطوة
1	جَهِّز VPS (2 vCPU / 4GB RAM كافية للبداية)
2	ثَبِّت Docker و Docker Compose
3	وَجِّه النطاق (سجل A) للخادم
4	أُنشِئ docker-compose.yml (n8n + Postgres + Caddy)
5	أُنشِئ .env بالمتغيرات — ⚠️ وُلِدَ N8N_ENCRYPTION_KEY عشوائيًا واحفظه خارجيًا
6	شَغِّل واختبر الوصول عبر HTTPS
7	فَعِّل جدار الحماية (اسمح 80/443 فقط)
8	أَعِدْ النسخ الاحتياطي واختبر الاستعادة
9	أَعِدْ المراقبة

🔒 الخطوة 8 ليست اختيارية، والكلمة المهمة فيها "اختبر". نسخة احتياطية لم تُخْتَبَر استعادتها ليست نسخة احتياطية — بل أمل. جَرِّب الاستعادة على خادم اختباري قبل أن تحتاجها.

⚙️ ما الذي تنسخه احتياطيًا؟

العنصر	الأهمية
قاعدة بيانات Postgres	🔴 حرج — تحوي ال workflows وال credentials والتاريخ
N8N_ENCRYPTION_KEY	🔴 حرج — بدونها النسخة عديمة الفائدة
ملفات الإعداد	🟡 مهم
بيانات ثنائية مخزنة	🟡 حسب الاستخدام

⚠️ **الخطأ القاتل الشائع:** نسخ قاعدة البيانات ونسيان مفتاح التشفير. النتيجة عند الكارثة: تستعيد قاعدة البيانات، فتجد كل ال credentials مشفرة وغير قابلة للفك. النظام يبدو موجودًا لكنه لا يعمل.

14.3 Queue Mode والتوسع

🔴 مُتَحَقِّق من توثيق n8n.

الحل	المعرض
Queue Mode + Workers	التنفيذات تتراكم في الطابور
Queue Mode (فصل التنفيذ عن الواجهة)	الواجهة تبطئ أثناء التنفيذ
Workers متعددة	تنفيذات طويلة تحجب غيرها

البنية⚙️

```
[Main] ← Webhooks الواجهة + الجدولة + استقبال
|
▼
[Redis] ← طابور المهام
|
├─ [Worker 1]
├─ [Worker 2]
└─ [Worker 3]      docker compose up -d --scale n8n-worker=3
```

الشروط الإلزامية (مُتحققة)⚙️

التفصيل	الشروط
على العملية الرئيسية وكل العمال	EXECUTIONS_MODE=queue
⚠️ الخطأ الأشهر: Could not find encryption key	N8N_ENCRYPTION_KEY متطابق
⚠️ SQLite لا تدعم الكتابة المتزامنة وستتلف البيانات	Postgres إلزامي
QUEUE_BULL_REDIS_PORT و QUEUE_BULL_REDIS_HOST	Redis

🐛 تشخيص: ظهور Could not find encryption key في العمال يعني أن N8N_ENCRYPTION_KEY غير متطابق بين الخدمات. هذا هو السبب الأول والأشهر.

⚠️ لا تنتقل لـ Queue Mode قبل الحاجة الفعلية. يضيف تعقيدًا حقيقيًا (Redis + عمال + إدارة). حسن workflows أولاً: أزل الحلقات غير الضرورية · فعّل Pagination · قلّل الاستدعاءات الخارجية · احذف بيانات التنفيذ القديمة. معظم مشاكل الأداء سببها تصميم سيئ لا نقص موارد.

Definition of Done — Module 14 ✓

- أختار بين Cloud و Self-hosted بمعايير واضحة. ☐

- ☐ نشر ن8n بـ Docker مع Postgres و HTTPS.
- ☐ N8N_ENCRYPTION_KEY محفوظ في مكان آمن خارج الخادم.
- ☐ اختبرت استعادة نسخة احتياطية فعليًا.
- ☐ أفهم شروط Queue Mode ومتى أحجازه.

Module 15 — Advanced Architecture

الهدف: تصميم أنظمة كاملة قبل بنائها. المدة: 180 دقيقة · المتطلبات: كل ما سبق

15.1 المشكلة: الـ Workflow العملاق

⚠️ الأعراض

الغرض	الأثر
+40 عقدة في مسار واحد	يستحيل فهمه
نفس المنطق مكرر في أماكن	تغيير واحد = تعديل خمسة مواضع
تعديل جزء يكسر آخر	خوف من التعديل
التشخيص يستغرق ساعات	تكلفة صيانة عالية
لا يمكن اختبار جزء منفردًا	اختبار الكل أو لا شيء

✓ الحل: المعيارية (Modularity)

قبل - نظام واحد متشابك ✕
[متابعة] → [إشعار] → [قرار] → [AI] → [تقييم] → [تخزين] → [تحقق] → [Webhook]
(عقدة في مسار واحد 40)

بعد - وحدات مستقلة ✓
[تخزين] → [تحقق] → [Webhook]
|
└─> استدعاء: Lead Scoring (Sub)
└─> استدعاء: AI Classification (Sub)
└─> استدعاء: Notification (Sub)

Sub-workflows 15.2

v1.2 executeWorkflowTrigger المشغل (each / once) (الأوضاع): v1.3 n8n-nodes-base.executeWorkflow

متى تفصل وحدة؟💡

المعيار	مثال
يُستخدم في أكثر من مكان	إرسال إشعار • تسجيل نشاط
مسؤولية واحدة واضحة	تقييم عميل
قد يتغير مستقلاً	قواعد التقييم
يحتاج اختباراً منفرداً	منطق معقد
يستدعي خدمة خارجية	عزل التغيير في مكان واحد

الوضع ⚙️

الوضع	السلوك	متى
once	كل ال items في تنفيذ واحد	معالجة جماعية
each	تنفيذ منفصل لكل item	عزل كل سجل عن الآخر

⚠️ each مع 1000 1000 item تنفيذ. انتبه للأداء وحدود الحصة.

★ تصميم واجهة الوحدة

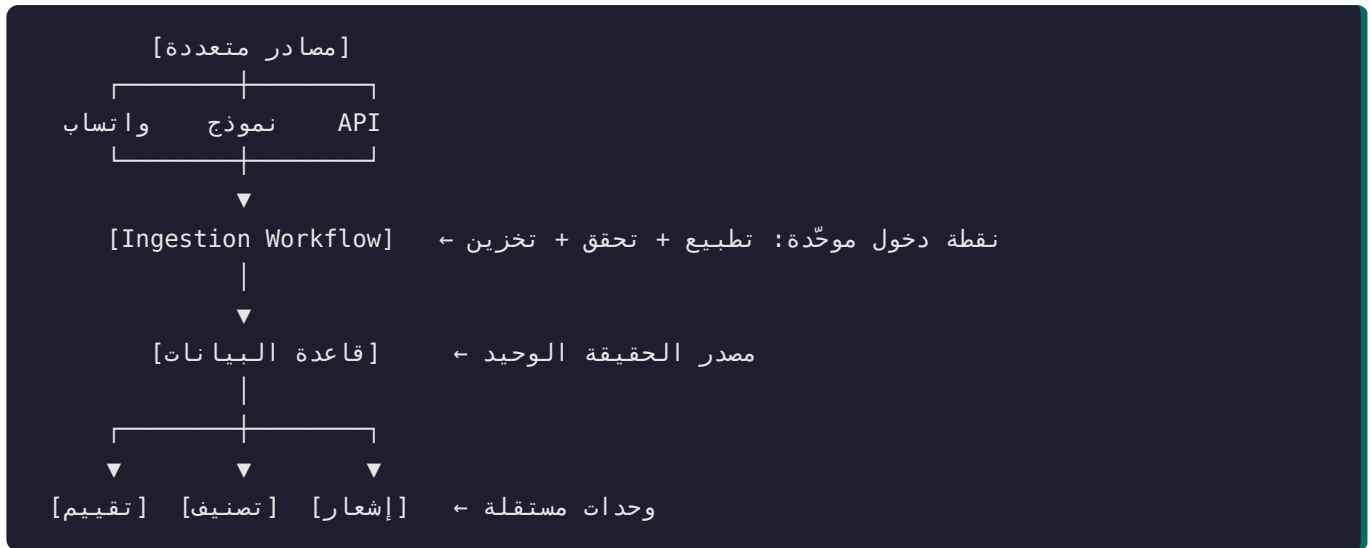
عامل ال sub-workflow كدالة برمجية: مدخلات واضحة، مخرجات واضحة، مسؤولية واحدة.

Sub-workflow: Lead Scoring

المدخل: { lead_id, budget, source, company_size }
المخرج: { lead_id, score, temperature, reasons[] }
المسؤولية: يحسب النقاط فقط – لا يحفظ ولا يرسل ولا يقرر

🧠 مبدأ فصل الاهتمامات (Separation of Concerns): وحدة تحسب النقاط وتحفظ في القاعدة وترسل إشعاراً = ثلاث مسؤوليات = ثلاثة أسباب للتغيير. افصلها. فتصبح كل وحدة قابلة للاختبار والتعديل وإعادة الاستخدام باستقلال.
الفائدة العملية: أردت تغيير قناة الإشعار من تيليجرام لواتساب؟ تعطل وحدة الإشعار فقط — ولا تلمس منطق التقييم إطلاقاً.

15.3 المعمارية القائمة على الأحداث



💡 لماذا هذا التصميم أقوى؟

الميزة	التفصيل
إضافة مصدر جديد سهلة	تُطَبِّع البيانات وتحققها — بلا لمس المنطق
مصدر حقيقة واحد	قاعدة البيانات، لا حالة موزعة
وحدات مستقلة	فشل الإشعار لا يعني ضياع العمل
قابل للاختبار	كل وحدة على حدة
قابل للتوسع	كل وحدة تتوسع بمعزل

🧠 **المبدأ الأهم:** افصل الاستقبال عن المعالجة. الـ webhook يجب أن يفعل أقل شيء ممكن: تحقق → خزّن → رُدّ بسرعة. ثم تعالج بقية العمل بشكل غير متزامن.

لماذا؟ لأن المرسل عادةً لديه **timeout**. إن استغرقت معالجتك 30 ثانية، يظن المرسل أن الطلب فشل ويعيد الإرسال — فتحصل على بيانات مكررة، وتُصعّب المشكلة على نفسك.

15.4 أنماط معمارية عملية

النمط	المشكلة التي يحلها	التنفيذ
Ingestion Gateway	مصادر متعددة بأشكال مختلفة	workflow واحد يطبق ويحقق
Router	كل نوع يحتاج معالجة مختلفة	Switch → sub-workflows
Queue + Worker	معالجة ثقيلة أو محدودة المعدل	جدول حالة + workflow مجدول يسحب
Circuit Breaker	خدمة خارجية متعثرة	حالة في القاعدة توقف المحاولات مؤقتًا
Dead Letter Queue	سجلات فشلت نهائياً	جدول للفشل + مراجعة يدوية
Anti-Corruption Layer	API خارجي يتغير	Set node يطبق الشكل مباشرة بعد المصدر
Saga / Compensation	عملية متعددة الخطوات فشلت جزئياً	خطوات تراجع صريحة

Dead Letter Queue أكثر الأنماط قيمة وأقلها استخدامًا. بدلاً من فقدان السجلات الفاشلة نهائياً، تُخزّن في جدول `failed_items` مع سبب الفشل والبيانات الأصلية. الفائدة المزدوجة: لا تفقد بيانات · وتحصل على قائمة قابلة للتحليل تكشف الأنماط المتكررة للفشل. مع العملاء: القدرة على قول "لم نفقد أي سجل، وهذه قائمة الـ 12 التي احتاجت تدخلاً" تختلف جذرياً عن "يبدو أن بعض السجلات ضاعت".

15.5 عملية التصميم — 11 خطوة قبل أي بناء

لا تفتح n8n قبل إكمال هذه الخطوات. ساعة في التصميم توفر عشر ساعات في إعادة البناء. هذه ليست مبالغة — بل تجربة متكررة.

#	الخطوة	المخرَج
1	حلّل المشكلة	ما الألم الحقيقي؟ ما تكلفته الحالية؟
2	حدد المدخلات	المصادر · الشكل · الحجم · التكرار
3	حدد المخرجات	ما ينتج · أين يذهب · بأي شكل
4	حدد الخدمات	ما الأنظمة المشاركة؟
5	حدد الـ APIs	متاحة؟ حدودها؟ مصادقتها؟
6	صمّم قاعدة البيانات	الجدول · العلاقات · الفهارس · القيود
7	حدد المصادقة	لكل خدمة · لكل نقطة دخول
8	صمّم معالجة الأخطاء	ماذا يفشل؟ ماذا يحدث؟ من يُبلِّغ؟
9	صمّم الأمان	الأسرار · التحقق · الوصول · الخصوصية
10	ارسم المعمارية	مخطط يفهمه غير التقني
11	قسّم لمراحل	ما أصغر نسخة تُنتج قيمة؟

★ الخطوة 11 هي الفرق بين مشروع يُسلّم ومشروع يُهجّر. لا تبني النظام كاملاً دفعة واحدة. ابنِ الجزء الذي يُنتج قيمة أولاً، سلّمه، اجمع ملاحظات، ثم وسّع. ميزة تجارية: العميل يرى نتيجة خلال أسبوع بدل انتظار شهرين على وعد — وهذا يبني الثقة ويمنع إلغاء المشاريع.

Definition of Done — Module 15 ✓

- ☐ قسّمتُ نظاماً كبيراً إلى sub-workflows بمسؤوليات واضحة.
- ☐ صمّمتُ واجهة كل وحدة (مدخل/مخرج) قبل بنائها.
- ☐ طبّقتُ نمطين معماريين على الأقل.
- ☐ رسمتُ معمارية كاملة قبل البناء.
- ☐ قسّمتُ مشروعاً إلى مراحل تسليم.

كيف يفكر المحترف

هذا القسم ليس معلومات — بل طريقة تفكير. ارجع إليه قبل كل مشروع.

من طلب العميل إلى نظام مُسلّم

المرحلة 1 — الاكتشاف (قبل أي وعد)

طلب العميل النموذجي:

"أبغى أتمّة استقبال العملاء والرد عليهم وتأهيلهم وتسجيلهم في قاعدة البيانات وإرسال إشعارات للفريق."

هذا ليس متطلبات — بل أمنية. مهمتك تحويله إلى مواصفات.

أسئلة الاكتشاف

المجال	الأسئلة
الحالي	كيف تعملون الآن؟ كم يستغرق؟ ما الذي يزعجكم أكثر؟
الحجم	كم عميلًا يوميًا؟ ما الذروة؟ ما النمو المتوقع؟
المصادر	من أين يأتون؟ (اطلب عينة حقيقية من كل مصدر)
التأهيل	ما الذي يجعل العميل "مؤهلاً" بالضبط؟
الرد	ماذا نرد؟ من يعتمد الرد؟
الإشعار	من يُبلّغ؟ بأي قناة؟ ما مدى الاستعجال؟
الأنظمة	ما الموجود لديكم؟ من يديره؟
النجاح	كيف نعرف أن المشروع نجح؟ (رقم محدد)
الفشل	ماذا يحدث إن أخطأ النظام؟ ما التكلفة؟

⚠ لا تسرّ قبل الاكتشاف. "أبغى أتمّة العملاء" قد تعني نظامًا بسيطًا بيومين، أو منصة كاملة بشهرين. التسعير قبل الفهم يعني إما خسارتك أو خسارة العميل — وكلاهما يضر السمعة.

🔗 السؤال الأقوى: "كيف نعرف أن المشروع نجح؟" إن لم يستطع العميل الإجابة برقم، فهو لا يعرف ما يريد بعد — وساعدته على تحديده جزء من قيمتك، ويميزك عمّن يبني ما يُطلب حرفيًا.

المرحلة 2 — الأسئلة الاثنا عشر

قبل أي بناء (من Module 1، ونكررها لأهميتها):

1. ما الحدث الذي يبدأ العملية؟
2. ما شكل البيانات الداخلة؟ (عينة حقيقية)
3. ما شكل المخرج المطلوب؟
4. ماذا لو كانت البيانات ناقصة؟
5. ماذا لو فشل الـ API؟
6. ماذا لو وصل الحدث مرتين؟
7. ما الحجم المتوقع؟
8. أين تُخزّن البيانات ومن يصل إليها؟
9. كيف أحمي الأسرار؟
10. كيف أعرف أن النظام توقف؟
11. كيف أختبر دون بيانات حقيقية؟
12. هل يمكن تبسيط المعمارية؟

المرحلة 3 — أسئلة مراجعة التصميم

السؤال	إن كانت الإجابة سيئة
هل هذا قابل للتوسع؟	أعد التصميم قبل البناء
هل فيه نقطة فشل واحدة؟	أضف مسارًا احتياطيًا
ماذا لو فشل الـ API؟	أضف retry + fallback + تنبيه
ماذا لو تكرر الـ webhook؟	أضف idempotency
كيف أتعامل مع Rate Limits؟	صمم التباطؤ الآن لاحقًا
أين أأخذ البيانات؟	اختر بمعايير لا بالعادة
كيف أحمي الأسرار؟	Credentials + متغيرات بيئية
كيف أراقب النظام؟	Error Workflow + Heartbeat
كيف أختبر؟	Pin Data + بيئة اختبار
هل يمكن تبسيطه؟	احذف كل ما ليس ضروريًا
هل هناك تكلفة غير ضرورية؟	راجع استعدادات AI والتنفيذات
هل هناك طريقة أوثق؟	فكر قبل أن تلتزم

🧠 السؤال العاشر هو الأصعب على المهندسين، والأهم. البساطة ليست نقص قدرة — بل قمة النضج الهندسي. النظام الذي يفهمه غيرك ويصونه بسهولة أفضل من النظام الذكي الذي لا يفهمه أحد سواك.

المشاريع العملية

التفاصيل الكاملة في مجلد `projects/`.

#	المشروع	يستخدم	المستوى	الملف
1	Form → Sheets → Email	M1–M3	Beginner 🏅	projects/01-form-to-sheets.md
2	Webhook → API → DB	M4–M7, M9	Intermediate 🥈	projects/02-webhook-api-database.md
3	Gmail → AI → تصنيف → مسودة رد	M10	Intermediate 🥈	projects/03-ai-email-assistant.md
4	Lead → تأهيل DB → AI → إشعار	M9–M12	Advanced 🥇	projects/04-lead-qualification.md
5	نظام أتمتة محتوى	M10, M15	Advanced 🥇	projects/05-content-automation.md
6	Capstone — نظام عميل كامل	كل شيء	Architect 🏰	projects/06-capstone.md

كل مشروع يحتوي على: تحليل المتطلبات · المعمارية · المخطط · العقد · ال Credentials · التنفيذ · الاختبار · معالجة الأخطاء · الأمان · التحسين · قائمة الإنتاج.

Capstone Challenge

👑 هذا اختبار رتبة Architect. لا يوجد حل جاهز في البداية.

السيناريو

شركة تصلها عملاء محتملون من عدة مصادر (نموذج الموقع، واتساب، إنستغرام، معارض). تريد: جمعها في مكان واحد • تصنيفها بالذكاء الاصطناعي • إرسالها للـ CRM • إشعار فريق المبيعات • متابعة حالة العميل تلقائيًا.

المطلوب منك — بالترتيب

#	التسليم
1	تحليل المشكلة — الألم الحقيقي وتكلفته الحالية
2	وثيقة متطلبات — من الاكتشاف، لا من الافتراض
3	معمارية — مخطط يفهمه غير التقني
4	التكاملات — ما الخدمات ولماذا هذه تحديدًا
5	مخطط قاعدة البيانات — جداول وعلاقات وفهارس وقيود
6	بناء الـ workflows — معياري لا عملاق
7	طبقة AI — بمخرج منظّم وبوابة ثقة
8	معالجة الأخطاء — Error Workflow + Heartbeat + DLQ
9	الأمان — قائمة Module 12 كاملة
10	اختبار — السيناريوهات السبعة
11	توثيق — يمكن غيرك من الصيانة

الحل النموذجي والمعمارية المرجعية في `projects/06-capstone.md` . ⚠ لا تفتحه قبل محاولة جادة. قيمة هذا التمرين كلها في المحاولة — النظر للحل مبكرًا يحوِّله إلى قراءة سلبية بلا فائدة.

Glossary — قاموس المصطلحات

المصطلح	بالعربية	التعريف
Workflow	سير عمل	مجموعة عقد متصلة تنفذ عملية
Node	عقدة	وحدة تنفذ مهمة واحدة
Trigger	مشغل	العقدة التي تبدأ التنفيذ
Item	عنصر	وحدة بيانات واحدة تنتقل بين العقد
Execution	تنفيذ	مرة واحدة اشتغل فيها الـ workflow
Credential	بيانات اعتماد	مفاتيح دخول مخزنة مشفرة
Expression	تعبير	كود ديناميكي بين <code>{{ }}</code>
Webhook	—	عنوان يستقبل طلبات ويبدأ التنفيذ
API	واجهة برمجية	طريقة تخاطب الأنظمة
REST	—	نمط شائع لتصميم APIs
Endpoint	نقطة نهاية	عنوان محدد في API
Payload	حمولة	البيانات المُرسلة
Header	ترويسة	بيانات وصفية للطلب
Rate Limit	حد المعدل	أقصى طلبات مسموحة في فترة
Pagination	ترقيم الصفحات	تقسيم النتائج لصفحات
OAuth 2.0	—	بروتوكول تفويض بلا مشاركة كلمة المرور
Idempotency	الحيادية التكرارية	تكرار العملية يُنتج نفس النتيجة
CRUD	—	إنشاء · قراءة · تحديث · حذف
UPSERT	—	أدخل أو حدّث إن وُجد
Primary Key	مفتاح أساسي	معرف فريد لكل صف
Foreign Key	مفتاح أجنبي	يربط جدولاً بآخر

المصطلح	بالعربية	التعريف
Index	فهرس	بنية تسرّع الاستعلامات
SQL Injection	حقن SQL	ثغرة تنفيذ أوامر عبر المدخلات
LLM	نموذج لغوي كبير	نموذج يولّد نصًا
Token	رمز	وحدة نص وتسعير في النماذج
Prompt	أمر	التعليمات المُعطاة للنموذج
Temperature	درجة العشوائية	0 = ثابت 1 = إبداعي
Structured Output	مخرج منظم	إجبار المخرج على صيغة محددة
Hallucination	هلوسة	اختراع معلومات بثقة
AI Agent	وكيل ذكي	نموذج يخطط ويستخدم أدوات
RAG	توليد معرّز بالاسترجاع	جلب مقاطع ذات صلة قبل التوليد
Embedding	تضمين	تمثيل رقمي لمعنى النص
Vector Database	قاعدة متجهية	تخزين وبحث في المتجهات
Sub-workflow	سير عمل فرعي	workflow يُستدعى من آخر
Queue Mode	وضع الطابور	تشغيل موزّع بعمّال
Worker	عامل	عملية تنقذ مهام من الطابور
DLQ	طابور الرسائل الميئة	مخزن للسجلات الفاشلة نهائيًا
Circuit Breaker	قاطع الدائرة	إيقاف مؤقت للمحاولات عند تعثر خدمة
Heartbeat	نبضة	فحص دوري يكشف توقف النظام
Fair-code	—	ترخيص n8n: مفتوح مع قيود تجارية

Final Checklist

قبل تسليم أي نظام لعميل

المتطلبات

- [] وثيقة متطلبات مكتوبة ومعتمدة من العميل
- [] معايير نجاح محددة برقم
- [] النطاق واضح وما هو خارج مكنوب صراحةً

المعمارية

- [] مخطط معماري يفهمه غير التقني
- [] مخطط قاعدة بيانات موثّق
- [] وحدات مفسّمة بمسؤوليات واضحة
- [] لا نقطة فشل واحدة حرجة

التنفيذ

- [] وعقدة workflow أسماء واضحة لكل
- [] workflow وصف لكل
- [] عند القرارات غير البديهية Sticky Notes
- [] لا منطق مكرر
- [] Pin Data منزوع

الموثوقية

- [] الإنتاج workflows مرتبط بكل Error Workflow
- [] على الأخطاء العابرة فقط Retry
- [] مطبّقة Idempotency
- [] للفشل النهائي Dead Letter Queue
- [] يكشف الصمت Heartbeat

الأمان

- [] لا أسرار في العقد
- [] مؤمنة webhooks
- [] استعلامات معاملية
- [] أقل صلاحية ممكنة
- [] لا أسرار في السجلات
- [] نموذج العمل متوافق مع الرخصة

الاختبار

- [] المسار الناجح
- [] بيانات ناقصة
- [] بيانات خاطئة النوع
- [] API فشل
- [] Timeout
- [] طلب مكرر
- [] حجم كبير



المراقبة

- [] تنبيهات مضبوطة
- [] سجلات كافية
- [] العميل يعرف كيف يتحقق من الحالة



التسليم

- [] دليل تشغيل
- [] كيفية التعديل الشائع
- [] جهة الاتصال عند المشاكل
- [] النسخ الاحتياطي مُعدّ ومُختبَر
- [] اتفاق الصيانة واضح

التقييم الذاتي النهائي

السؤال	إن كانت الإجابة "لا"
أستطيع بناء workflow بسيط من الصفر بلا مرجع؟	راجع M1-M3
أشرح تدفق البيانات في أي workflow؟	راجع M4
أكتب تعبيرًا معقدًا بنفسى بلا نسخ؟	راجع M5
أشخص خطأ لم أره من قبل بلا AI؟	راجع M11 + Debugging Lab
أربط API لم أستخدمه من قبل؟	راجع M7
أصمم مخطط قاعدة بيانات صحيحًا؟	راجع M9
أبني AI Agent بمخرج منظم؟	راجع M10
نظامي يتعامل مع الفشل والتكرار؟	راجع M11
أمنّث كل نقاط الدخول؟	راجع M12
نشرت نظامًا إنتاجيًا بنسخ احتياطية مُختبرة؟	راجع M14
أحوّل طلب عميل غامض إلى معمارية موثقة؟	راجع M15 + كيف يفكر المحترف

👑 إن كانت كل الإجابات "نعم" — فأنت Automation Architect. والاختبار الحقيقي ليس هذه القائمة، بل: عميل حقيقي، ومشكلة حقيقية، ونظام يعمل بعد ستة أشهر دون تدخلك.

[templates/](#) · [cheat-sheets/](#) · [exercises/DEBUGGING_LAB.md](#) · [projects/](#) نهاية المنهج الرئيسي. تابع في: